Movie Ticketing System

Software Requirements Specification

Version-1

02-05-2026

Group #5
Felipe Muñoz
Miriam Valenzuela
Victor Lopez

Prepared for
CS 250 - Introduction to Software Systems
Instructor: Dr. Gus Hanna
Fall 2024

Revision History

Date	Description	Author	Comments
02-12-26	Version 1 - Setting Project Requirements	Group 5	First Revision - Requirements Specification
02-05-26	Version 2 - Software Design Specification	Group 5	Second Revision - Software Design Specification
03-19-26	Version 3 - Test Plan	Group 5	Third Revision - Test Plan
04-09-26	Version 4 - Data Management	Group 5	Fourth Revision - Data Management

Document Approval

The following Software Requirements Specification has been accepted and approved by the following:

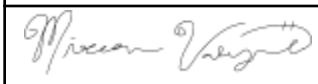
Signature	Printed Name	Title	Date
	<Your Name>	Software Eng.	
	Dr. Gus Hanna	Instructor, CS 250	
	Miriam Valenzuela	Computer Science Undergraduate	02/19/2026
	Victor Lopez	Computer Science Undergraduate	02/19/2026
	Felipe Munoz	Computer Science Undergraduate	02/19/2026

Table of Contents

1. Introduction

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for the Movie Theater Web Ticketing System, a browser-based system that allows customers to browse movies and showtimes, view reviews, select seats, purchase tickets (including discounted ticket types), and submit feedback. The system also provides administrative capabilities for managing showtimes, ticket rules, and reviewing security/feedback logs. This document is written to provide sufficient detail for a software engineer to design, implement, and test the system in a manner consistent with the following stated requirements.

1.1 Purpose

The purpose of this SRS is to formally specify the requirements of the Movie Theater Web Ticketing System, including interfaces, and quality attributes. The intended audience/stakeholders includes:

- Course stakeholders: Instructor for grading and requirement validation.
- Client representatives: Student "clients" who provided interview input for expected behavior.
- Development Team: Software Engineers/Designers implementing the system.
- Test Team: Students responsible for creating test plans and validating requirement compliance.
- Project Managers: Individuals tracking scope, priorities, and deliverables.

1.2 Scope

1.2.1 Product Identification

The software product to be produced is the Movie Theater Web Ticketing System (MTWTS).

1.2.2 What the Product Will Do

The MTWTS will:

- Provide a web browser-based interface for customers to:
    - Browse currently available movies and view details.
    - View showtimes and ticket availability.
    - View reviews and critic quotes gathered from online review sources.
    - Select seats and purchase tickets (including multiple discount ticket types).
    - Submit customer feedback.
- Enforce purchasing rules including:
    - A maximum of 20 tickets per transaction.
    - Ticket purchasing availability from 14 days prior to showtime until 10 minutes after showtime begins.
- Provide security controls to reduce automated/bot bulk purchasing.
- Provide an Administrator Mode for managing showtimes and ticket configuration, and for reviewing security and feedback logs.

- Interface with a theater showtime/ticket inventory database to ensure accurate availability and prevent double-selling.

1.2.3 What the Product Will Not Do

The MTWTS will not:

- Provide a native mobile application (iOS/Android).
- Support ticket purchases outside of the defined time window or beyond the 20-ticket transaction limit.
- Define or control external third-party review sites' content availability, the system will display reviews when the retrieval is possible and show a fallback message when not available.
- Implement reasonable protections to reduce bot activity and maintain availability for legitimate users.

1.2.4 Benefits, Objectives, and Goals

The primary objectives and benefits are:

- Improve customer access and convenience by enabling ticket purchases through a web browser without installation.
- Maintain accurate inventory by integrating with the showtime/ticket database and preventing seat double-booking.
- Support high-demand events by handling at least 1000 concurrent users while enforcing fair purchasing rules.
- Reduce automated abuse through bot prevention mechanisms.
- Increase customer engagement by displaying movie reviews/critic quotes and providing a feedback system for service improvement.
- Enable theater operations by providing an administrator mode to manage showtimes and ticket options.

1.3 Definitions, Acronyms, and Abbreviations

- SRS: Software Requirements Specification
- MTWTS: Movie Theater Web Ticketing System
- UI: User interface
- Admin / Administrator: Authorized user with access to administrator-only features
- Showtime: A scheduled screening of a movie at a specific date/time and auditorium
- Ticket Inventory: Real-time record of available/sold seats for a showtime
- Checkout: Process where the user confirms ticket selection and complete payment
- Rate Limiting: Restricting request frequency to reduce abuse or automated activity
- Bot: Automated software attempting to perform actions
- Human Verification Challenge: A step intended to confirm a real user
- Discount Ticket Type: A ticket category with adjusted pricing
- TLS/HTTPS: Secure encrypted communication protocol used for web traffic

1.4 References

Arms, W. Y. (n.d.). *Scenarios and use cases* [Lecture slides]. Canvas. https://sdsu.instructure.com/courses/197506/files/19593982?module_item_id=5531943

Burak, A. (2020, September 10). *How to Write a Software Requirement Specification (SRS)*. Relevant.software. https://relevant.software/blog/software-requirements-specification-srs-document/

Group 5. (2026, February 12). *Client interview notes, Group 5*. Collected during in-class interviews.

Hanna, G. (2024). *Software requirements specification template* [Template]. CS 250: Introduction to Software Systems. https://sdsu.instructure.com/courses/197506/files/19593981?module_item_id=5531950

Institute of Electrical and Electronics Engineers. (1998). *IEEE recommended practice for software requirements specifications* (IEEE Std 830-1998).

Krüger, N. (2023, January 17). *How to write a software requirements specification (SRS document) | perforce software*. Perforce Software. https://www.perforce.com/blog/alm/how-write-software-requirements-specification-srs-document

1.5 Overview

The remainder of this SRS is organized to present the system requirements in a structured, testable form:

- Section 2 describes the overall system perspective, product functions, user classes, operating environment, assumptions, and constraints.
- Section 3 defines the specific requirements, including external interfaces, functional requirements organized by major features and use cases, data requirements, design constraints, and non-functional requirements (performance, usability, security, etc.).

- Section 4 lists the analysis models used to help derive, clarify, and validate the requirements in Section 3. Each model includes an introduction, narrative description, and traceability to relevant SRS requirements.
- Section 5 defines how requirement changes will be proposed, reviewed, approved, tracked, and incorporated into the updated versions of this SRS. It identifies who may submit change requests, the submission method, and how the team and client/instructor will approve and document changes to maintain version control.
- Appendices provide supporting information that may help interpret or implement the requirements.

2. General Description

The MTWTS is a browser based platform designed for customers to browse movies and their showtimes, select seats, purchase tickets from varying discount type categories, and submit customer feedback. The system additionally provides Administrative roles that allow for managing of showtimes, ticket configuration, and reviewing of system security and feedback logs. The system is built for varying users from customers to movie theater staff , while operating reliably under high demand events and preventing automated abuse.

2.1 Product Perspective

The MTWTS uses various systems that create a broader ecosystem of external services that it depends on to ensure usability for users. The system interfaces with the theater's showtimes and ticketing inventory database , retrieving necessary movie listings, showtimes, auditorium, seat maps, and real time availability. Connection to online review sources shall allow for the retrieval of critic quotes and review summaries used to display for the customers. Integration to a third-party payment processor will allow for the secure handling of all transactions. Additionally, email and SMS delivery services will be used to complete actions such as user verifications and booking confirmations.

2.2 Product Functions

User Account Management**:** Customers will be able to register accounts, login , and access features such as order history, account information, and saved payment methods.

Movie Browsing and Showtimes: Customers can browse through different movies and their showtimes, showcasing each movie's summary, trailer, ratings, runtime, cast along with ticket availability. As well as showing online critic quotes and review summaries.

Ticket and Seat Selection: Customers can select seats from the given seat map , which will have the options of varying discount categories and ticket types. The purchase of the tickets will be done through a secure checkout. The system will enforce purchasing eligibility and transaction limits.

Bot Abuse & High Demand Protection: The system will monitor for suspicious activity to prevent automated abuse, as well as ensure that legitimate users have fair access to high demand tickets .

Customer Feedback: Customers will be able to submit feedback ratings and written text.

Administrator Mode: Authorized users will be allowed to manage different movie details and settings, as well as allowing for review of feedback and logs.

Concession Pre-Order: Customers can pre-order concessions ahead of their showtimes, with specified pick up instructions.

2.3 User Characteristics

Guest Users: These users will vary widely in the user base as they can be anyone from young users to senior users. Therefore since there is a wide range in the technical experience and education there should be no specialized knowledge required to use the system.

Register Users: These users will have a similar profile as they are more acquainted with the system , therefore allowing for more features such as account management and order history.

Administrators: The users in the administration will have moderate to high level experience with the system as they are managing different settings within the system. They will be comfortable with higher level knowledge of content management. (possibly add different levels of administrators)

2.4 General Constraints

Regulatory Policies: The system will comply with any applicable data protection and consumer privacy laws, including the handling of secure payment processes (no raw credit card data) .

Interfaces to Other Applications: The system will be constrained by the other interfaces that it connects with in terms of availability, capability, and terms that other services may apply. Additionally, the system will have to be delivered through the standard browser.

Parallel Operations: The system has to handle multiple concurrent users, leading to constraints where many users may be completing the same action simultaneously without having any conflicts in data integrity.

Audit Functions: The system will be maintaining security and activity logs to keep the software running smoothly , constraining how some events will have to be handled and logged.

Reliability Requirements: The system should remain available under high traffic conditions.

Critical of The Application: The system will have features such as seat reservation and payments(like pre-order concessions) where constraints of failure show up and handling of errors will be crucial for both the theater and the user.

Safety and Security Considerations: The system will have protections against automated abuse and other security issues such as session vulnerabilities. These lead to the constraints of the user authentication and session management.

2.5 Assumptions and Dependencies

Assumptions:

- Reliable Internet Access
- Accurate and up to date information for showtime, seat configuration, pricing
- Valid Email / Phone Number
- Online review sources accessible

Dependencies:

- Third-Party payment processor , availability and reliability (unexpected downtimes or changes)
- Theater's Showtime and Ticket Inventory Database being available and accurate
- Email & SMS delivery , crucial for bookings, user verification

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

UI-1. Web Browser UI (P1)

- UI-1.1 The system shall be usable from a modern web browser (Chrome, Firefox, Safari, Edge) without requiring installation of a mobile app. (P1)
- UI-1.2 The system shall provide the following customer pages: Home, Movie, Details, Showtimes, Seat Selection, Cart/Checkout, Order Confirmation, Order History, Customer Feedback. (P1)
- UI-1.3: The system shall provide an Admin mode UI accessible only to authorized administrators. (P1)
- UI-1.4 The system shall display ticket purchase eligibility rules (time window + ticket quantity limit) before checkout. (P1)
- UI-1.5 The system shall support accessibility features: keyboard navigation and readable contrast for all interactive elements. (P2)

3.1.2 Hardware Interfaces

HW-1. Client Device Support (P2)

- HW-1.1 The system shall run on devices capable of running a modern web browser (desktop/laptop/tablet). (P2)
- HW-1.2 No specialized hardware shall be required for customers. (P3)

3.1.3 Software Interfaces

SW-1. Showtime/Ticket Inventory Database Interface (P1)

- SW-1.1 The system shall interface with the theater's showtime and ticket availability database to read: movies, showtimes, auditorium, seat map, and ticket inventory. (P1)
- SW-1.2 The system shall reserve selected seats temporarily during checkout for a configurable time (e.g., 5 minutes) to prevent double selling. (P1)

SW-2. Online Review Sites Interface (P2)

- SW-2.1 The system shall retrieve and display critic quotes and review summaries from online review sources (via API when available, otherwise via scraping where permitted). (P2)
- SW-2.2 The system shall display a timestamp indicating when review data was last updated. (P3)

SW-3. Payment Processor Interface (P1)

- SW-3.1 The system shall integrate with a payment processor to process card payments. (P1)
- SW-3.2 The system shall not store raw credit card numbers. (P1)

3.1.4 Communications Interfaces

COM-1. Network Communications (P1)

- COM-1.1 The system shall use HTTPS for all communications between the browser and server. (P1)
- COM-1.2 The system shall timeout inactive sessions after a configurable duration (e.g., 30 minutes). (P2)

3.2 Functional Requirements

3.2.1 Movie Browsing & Showtimes

3.2.1.1 Introduction
Customers must be able to view movies, details, showtimes and reviews.

3.2.1.2 Inputs

- Search keywords (movie title, genre)
- Location/Theater selection (if applicable)
- Date selection

3.2.1.3 Processing

- FR-1.1 The system shall display a list of currently available movies. (P1)
- FR-1.2 The system shall allow customers to view movie details (summary, trailer, rating, runtime, cast). (P1)
- FR-1.3 The system shall display showtimes for a selected movie. (P1)
- FR-1.4 The system shall display ticket availability per showtime using the ticket database interface.
- FR-1.5 The system shall display critic quotes and review snippets for a movie. (P2)

3.2.1.4 Outputs
- Movie list, movie detail page, showtime list, review snippets.

3.2.1.5 Error Handling
- EH-1.1 If showtimes cannot be loaded, the system shall display an error message and allow retry. (P2)
- EH-1.2 If review sources are unavailable, the system shall show "Reviews currently unavailable" without blocking ticket sales. (P3)

3.2.2 Ticket Purchase & Eligibility Rules

3.2.2.1 Introduction
Customers must be able to select seats, apply discounts, and purchase tickets within defined rules.

3.2.2.2 Inputs
- Selected showtime
- Selected seats/ticket quantity
- Discount type (adult, child, senior, student, etc.)
- Payment info (via payment processor)

3.2.2.3 Processing
Purchase Limits/Timing:
- FR-2.1 The system shall restrict purchases to a maximum of 20 tickets per transaction. (P1)
- FR-2.2 The system shall allow ticket purchases only within 14 days before showtime and up to 10 minutes after showtime begins. (P1)
- The system shall prevent checkout if the selected showtime is outside the allowed purchase window. (P1)

Seat Selection & Inventory:
- FR-2.4 The system shall display the seat map for the selected showtime. (P1)

- FR-2.5 The system shall prevent a customer from purchasing seats that are no longer available. (P1)
- FR-2.6 The system shall confirm the reservation/purchase in the database after successful payment. (P1)

Discount Tickets:
- FR-2.7 The system shall support multiple ticket types/discounts (e.g., adult, child, senior, student). (P2)
- FR-2.8 The system shall compute total cost including discounts and taxes/fees before payment confirmation. (P1)

3.2.2.4 Outputs
- Order confirmation number
- Email/receipt confirmation
- Updated seat availability in database

3.3.3.5 Error Handling
- EH-2.1 If payment fails, the system shall not finalize the ticket sale and shall release reserved seats after the reservation timeout. (P1)
- EH-2.2 If the user exceeds the 20-ticket limit, the system shall show a clear message and block checkout. (P1)

3.2.3 Bot Prevention / High-Demand Protection

3.2.3.1 Introduction
The system must reduce automated/bot bulk purchasing.

3.2.3.2 Inputs
- User activity signals (rapid requests, repeated checkout attempts, suspicious patterns)

3.2.3.3 Processing
- FR-3.1 The system shall detect unusually high request rates from a single account or IP and apply rate limiting. (P1)
- FR-3.2 The system shall require an additional verification step (e.g., CAPTCHA) when bot-like behavior is detected. (P1)
- FR-3.3 The system shall block purchases when automated behavior is confirmed and log the event for admin review. (P2)
- FR-3.4 The system shall enforce automated abuse protections including rate limiting and bot detection on ticket purchase and checkout endpoints. (P1)

- FR-3.5 The system shall log security-related events (rate limit triggered, verification triggered/failed, blocked transaction) with timestamp, user/account identifier, and source IP for administrator review. (P2)

3.2.3.4 Outputs
- Verification prompt, blocked message, security log entry

3.2.3.5 Error Handling
- EH-3.1 If verification service is unavailable, the system shall fall back to rate limiting and manual retry messaging. (P2)

3.2.4 Administrator Mode

3.2.4.1 Introduction
Admins manage movies/showtimes/pricing and monitor suspicious activity.

3.2.4.2 Inputs
- Admin login credentials
- Showtime/movie updates
- Discount configurations

3.2.4.3 Processing
- FR-4.1 The system shall provide an administrator-only login area. (P1)
- FR-4.2 The system shall allow admins to create/update/remove showtimes. (P1)
- FR-4.3 The system shall allow admins to configure ticket types and discounts. (P2)
- FR-4.4 The system shall allow admins to review logs of blocked/flagged bot activity. (P2)

3.2.4.4 Outputs
- Updated showtime schedule, updated pricing/discount rules, admin reports/log views.

3.2.4.5 Error Handling
- EH-4.1 Unauthorized users attempting admin access shall be denied and the attempt shall be logged. (P1)

3.2.5 Customer Feedback System

3.2.5.1 Introduction
Customers can submit feedback about their experience.

3.2.5.2 Inputs
- Feedback rating (1-5)
- Feedback text
- Optional order number

3.2.5.3 Processing
- FR-5.1 The system shall allow customers to submit feedback. (P2)
- FR-5.2 The system shall store feedback entries with timestamp and (if provided) and order reference. (P2)
- FR-5.3 The system shall allow admins to view feedback submission. (P3)

3.2.5.4 Outputs
- Feedback confirmation message, admin feedback listing.

3.2.5.5 Error Handling
- EH-5.1 If feedback can not be submitted, the system shall preserve the typed message locally on the page until the user dismisses it or retries. (P3)

3.3 Use Cases

3.3.1 Use Case #1: Buy a Movie Ticket

Customers can search and find a movie and its corresponding showtime and location, and purchase a seat/ticket.

Primary Actor:
- Guest customer or Registered User

Secondary Actors:
- Showtime/Ticket inventory database interface

Preconditions:
- Customer using a supported web browser (UI-1.1).
- Movie showtimes and seats exist in database (SW-1.1, FR-1.1 through FR-1.4)

Trigger:
- Customer navigates to the home or search page

Success End Condition:
- The customer has selected a showtime and can proceed to the seat selection.

Success Timeline:
- The customer opens the website.
- The system displays the Home page and a list of available movies (FR-1.1).
- The customer can search by keyword and also use filters to narrow results.
- The system will display the matching movies and corresponding showtimes (FR-1.1, FR-1.3).
- The customer selects a movie.
- The system will display movie details such as the summary, trailer, rating, runtime and cast (FR-1.2).

- The customer selects the showtime for the movie.
- The system displays ticket availability (FR-1.3, FR-1.4).
- The system proceeds to Seat Selection (FR-2.4).

Failed End Condition

- Customer cannot proceed due to a system failure

Extensions

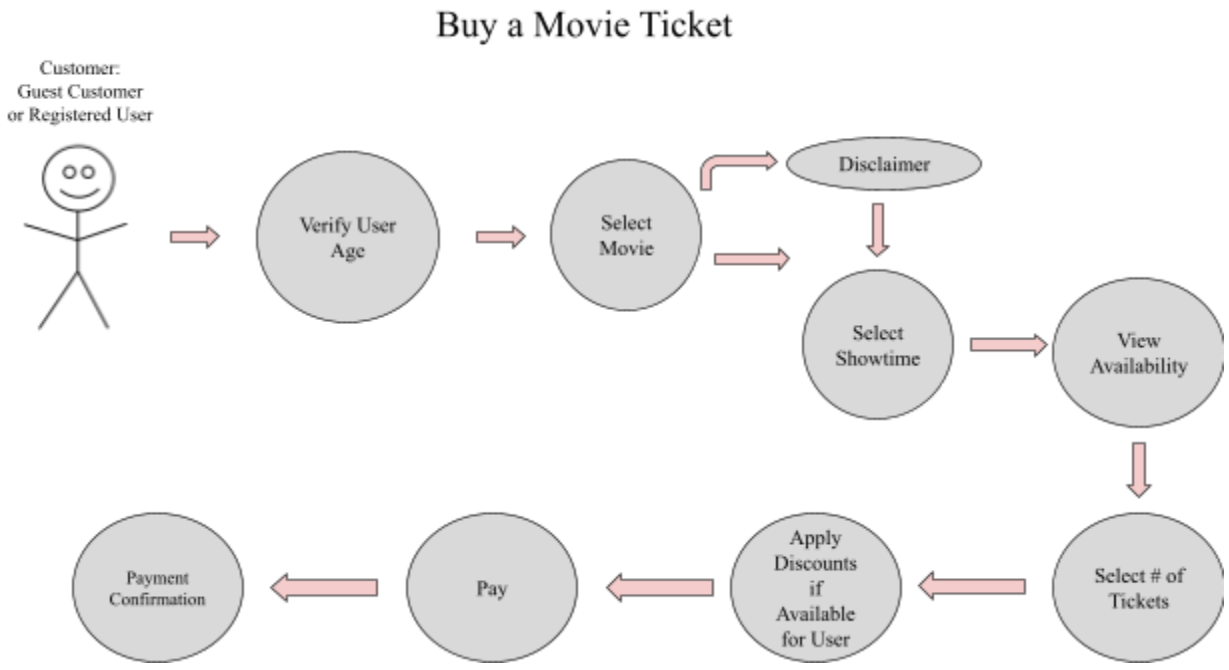
- If showtimes cannot be loaded the system displays a respective error message and prompts to either refresh to then retry (EH-1.1).
- If review sources are unavailable, the system shows “Reviews currently unavailable” without blocking the ticket sales (EH-1.2).

Traceability:

UI-1.1 and UI-1.2, SW-1.1, FR-1.1 through FR-1.5, EH-1.1 and EH-1.2

Priority:

P1



3.3.2 Use Case #2: Log In for Registered User

Allow a registered user to access their account securely to manage purchases, account information, order history, saved payment methods, resend tickets and extendable account features.

Primary Actor:

- Registered User

Secondary Actors:

- Authentication Service
- Email or SMS Service for verification

Preconditions:

- Users' accounts exist in the database.
- Users have access to their registered email and/or phone number (if verification is required).

Trigger:

- User log in menu

Success End Condition:

- Users are authenticated and have an active session.

Success Timeline:

- The user enters a username, address or phone number.
- The system securely encrypts/decrypts and validates credentials.
- Additional verification required, prompts user to either select their email or SMS
- A one time verification code via the selected method
- The Customer receives verification code and inputs it into a prompt
- The system verifies the code, logs the login event and grants access
- The system displays the customer dashboard with tabs/links to account features, purchases, news about ongoing sales/discounts.

Failed End Condition

- The user is not authenticated and thus no session is created.

Extensions

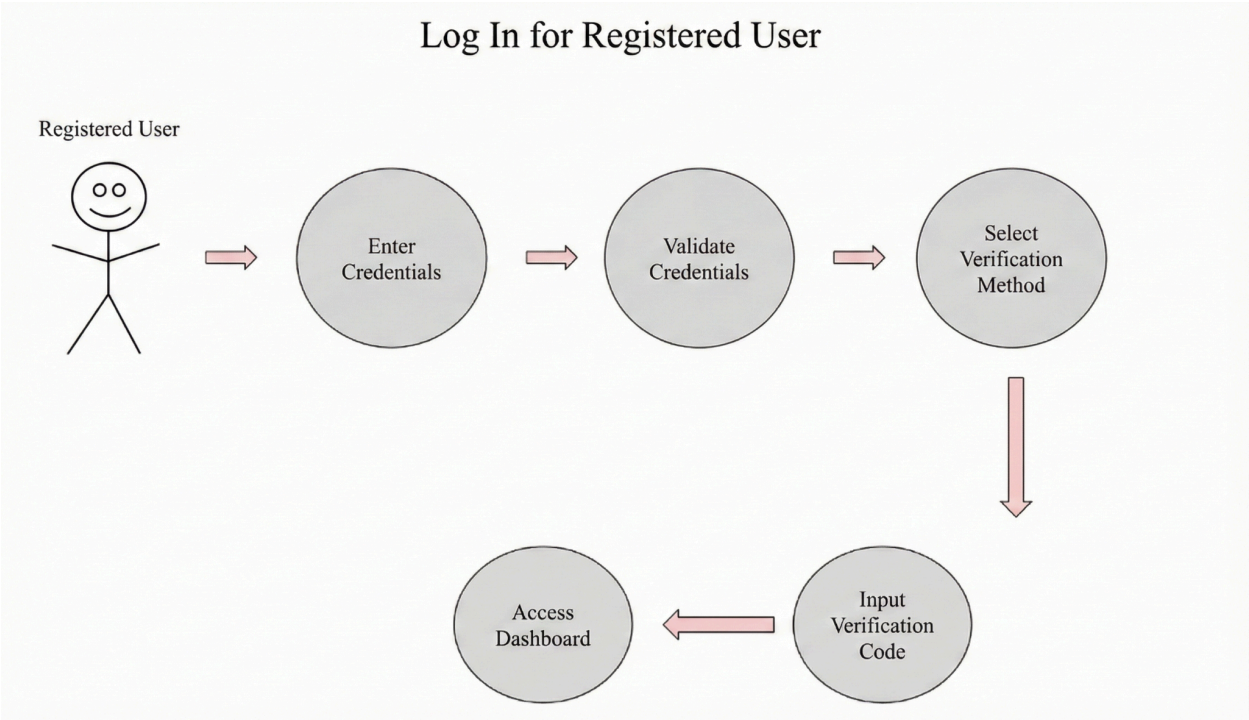
- If the credentials are invalid the system displays an error message and prompts the user to retry.
- If the verification code is incorrect (e.g misstyped) or expired, the system displays an error message and allows a new verification code to be re-sent, and re-try.

Traceability:

UI-1.2, COM-1.2

Priority:

P1



3.3.3 Use Case #3: Register an Account

A guest user creates an account to become a registered user.

Primary Actor:

- Guest user

Secondary Actors:

- Account Service
- Email or SMS service for verification

Preconditions:

- Guest user has no account in the database.
- Users have access to their registered email and/or phone number (if verification is required).

Trigger:

- Guest user selects the option to “Create/register Account”.

Success End Condition:

- The account exists and can be used to log in.

Success Timeline:

- The guest user opens the account creation page.
- The system displays the registration form.
- The guest enters required information such as either their username and password and may optionally include both their phone number and email address, but one of either is necessary for verification purposes.
- The system validates the fields and checks for duplicates.

- Guest selects email or SMS for verification, however if the option was not input for the account creation, it is only stored temporarily for verification purposes.
- The system sends a verification code to the selected method.
- The guest enters the verification code.
- The system verifies the code and creates the account.
- The system confirms the account creation and leads the now registered user to the homepage with the extended features.

Failed End Condition

- The account is not created.

Extensions

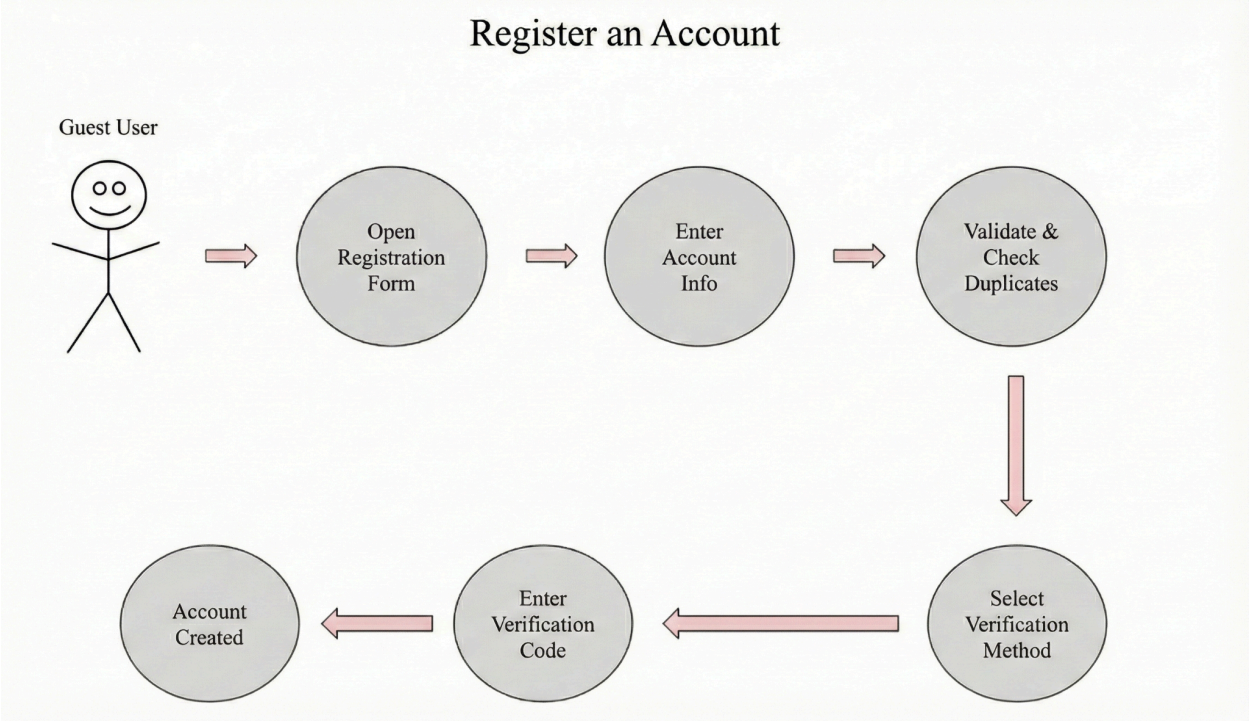
- If the email/username/phone-number already exists the system rejects the registration and requests a different value.
- If verification fails due to being incorrect or expired, the system allows resend and retry.
- If the guest provides only one contact method the system uses that method for verification and future contact as applicable but still allows the user to add additional contact info which can be used as well to log in for the username prompt.

Traceability:

UI-1.2

Priority:

P2



3.3.4 Use Case #4 Administrator Control Panel

Admins can manage movies, showtimes, pricing/promotions, seat-hold overrides/cancellations, user accounts, and view analytics and security logs of potential bot activity.

Primary Actor:
- System Administrator

Secondary Actors:
- Showtime/ticket inventory database interface.
- Authentication service.
- Logging/analytics service.

Preconditions:
- Admin is authorized to access the admin panel (UI-1.3).
- Admin has valid admin credentials (FR-4.1).
- The system and database interfaces are available (SW-1.1).

Trigger:
- Admin navigates to the Admin log in page (separate from the normal user log in page) and logs in.

Success End Condition:
- Requested administrator actions are completed successfully and recorded.

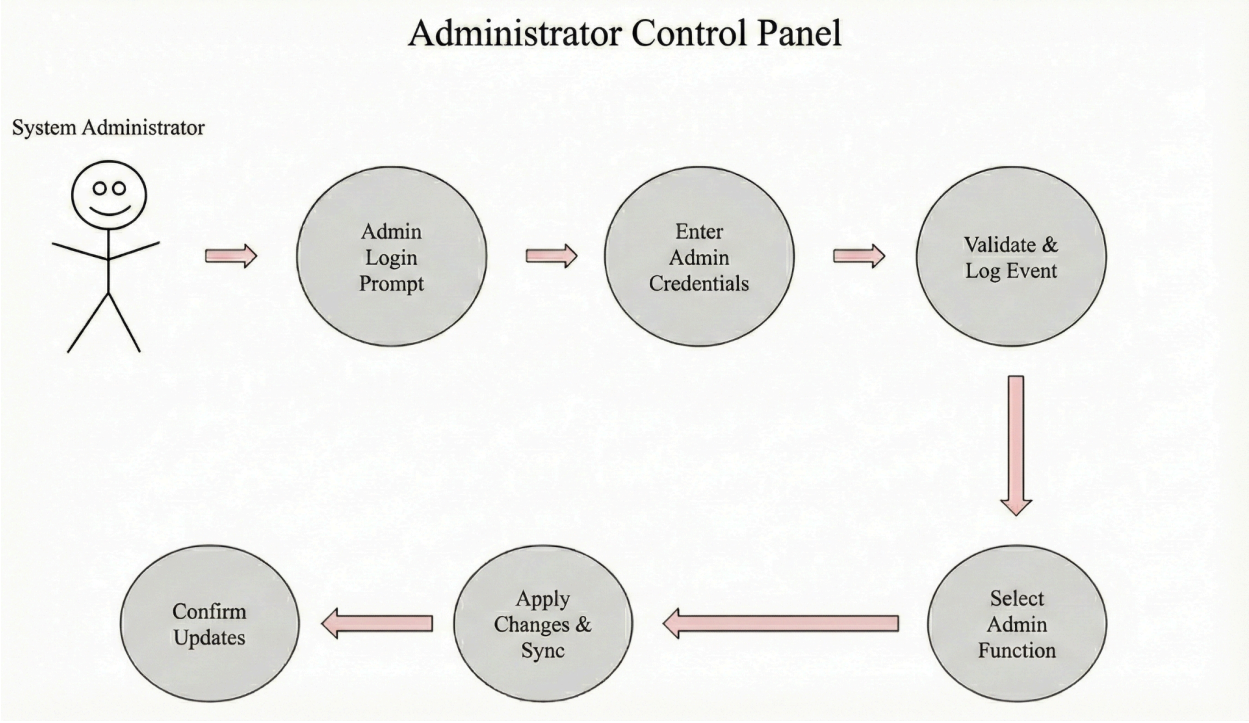
Success Timeline:
- Admin navigates to the Admin UI.
- The system displays the admin login prompt (FR-4.1).
- Admin enters valid admin credentials.
- The system validates credentials, logs the login event, and displays the Admin Control Panel (FR-4.1).
- Admin selects an administrative function such as manage movies/showtimes, update pricing/promotions, manage users, view logs/analytics.
- The system displays the relevant admin page and its corresponding data for the selected function (FR-4.2, FR-4.3, FR-4.4).
- Admin has this functionality:
    - Create, update and remove a movie or showtime (FR-4.2).
    - Configure ticket types, discounts, prices, and promotions (FR-4.3)
    - Manage user accounts with actions such as view, update and disable (FR-).
    - Override seat holds or perform cancellations when necessary (FR-).
    - View security logs and see suspicious activity events (FR-4.4)
- The system validates the requested changes, applies updates and syncs across the databases and services as well as logs the action for audit purposes (FR-4.4)
- The system displays confirmation of the completed actions and updated data.

Failed End Condition
- Admin cannot complete the action due to either an authorization error, validation error, or system failure and no changes applied.

Extensions
- Unauthorized access attempt from a non-admin or incorrect credentials, the system denies access and logs the attempt for auditing.
- If the admin while in Admin Control Panel submits invalid data such as impossible showtimes or pricing the system rejects the change and displays the appropriate error message.

- If the database or underlying services cannot be reached the system displays an error and does not apply and changes, prompting the admin to try later and an alert is set.

Traceability:
UI-1.3, FR-4.1 through FR-4.4, EH-4.1
Priority:
P1



3.3.5 Use Case #5 Purchase Food Ahead of Time

Guest and Registered Users can pre-order concessions for a selected showtime and receive confirmation for pickup.

Primary Actor:
- Guest or Registered user.

Secondary Actors:
- Payment processor
- Email service
- SMS service

Preconditions:
- User is using a supported browser (UI-1.1).
- User has selected a movie showtime (UC-1)
- Ticket purchase eligibility rules are displayed before checkout (UI-1.4).

Trigger:
- Customer selects the Order Food or Add Concessions during the ticket flow from the showtime page.

Success End Condition:

- Food order is successfully placed and the customer receives an order confirmation for pickup

Success Timeline:

- The customer selects the option to purchase food ahead of time.
- The system displays the concessions menu with the available items and quantities.
- The user selects the items and quantities to add to their order.
- The system displays an order summary with the total cost before the payment confirmation.
- The user provides the delivery contact info either their email, phone or notification to their menu for registered users for the receipt confirmation.
- The user submits their payment details and confirms their purchase order.
- The system processes the payment via the payment processor (SW-3.1).
- If the payment is approved the system displays a confirmation number and pickup instructions.
- The system sends the food order and confirmation via the customer's selected method.

Failed End Condition

- The food order is not finalized and the customer is given the corresponding error and troubleshooting message.

Extensions

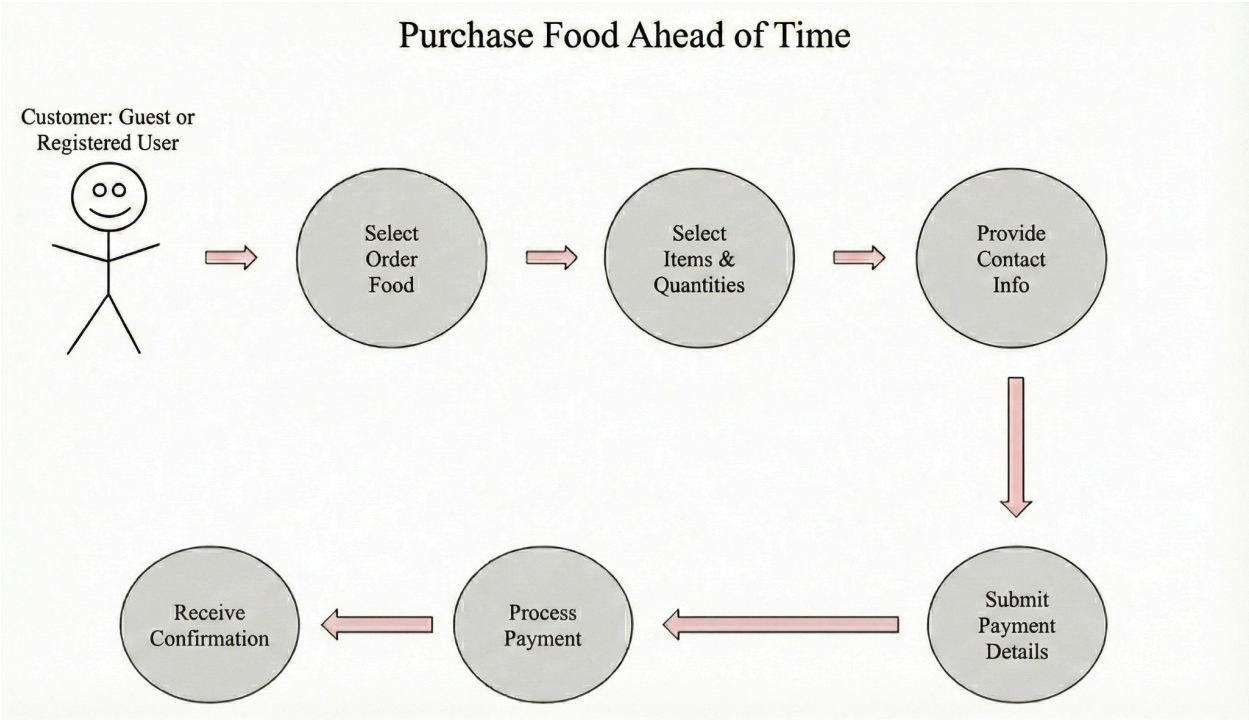
- If the payment fails the system does not finalize the food order and displays an error message and allows the user to retry (EH-2.1).
- If the customer attempts to order outside the allowed purchase timing rules for the associated showtime the system will block the checkout and display the eligibility rule message (FR-2.2, UI-1.4).
- If the customer exceeds the transaction limit rule when tickets and food are purchased together the system blocks the checkout and displays a message (FR-2.1, EH-2.2).

Traceability:

UI-1.1, UI-1.4, SW-3.1, SW-3.2, FR-2.1, FR-2.2, EH-2.1, EH-2.2

Priority:

P2



3.4 Classes / Objects

3.4.1 Movie

Movie
3.4.1.1 Attributes • movieId : int • title : string • summary : • trailer : string • rating : int • runtime : double • cast : list • genre : string
3.4.1.2 Functions • browse_movie(genre : string) : movie • view_movie_details(movieId : int) : movie • showtimes(movieId : int, Date : date) : showtime

3.4.2 Showtime

Showtime
3.4.2.1 Attributes • Showtime : string • movieId : int • Date : Date (class) • time : int • auditorium : int
3.4.2.2 Functions • display_showtimes(movieId : int, Date : Date) : Showtime • check_availability(Showtime : string, auditorium : int) : boolean

3.4.3 Seat

Seat
3.4.3.1 Attributes • seatId : string • seatNumber : int • reservation : boolean • showtimeId : string
3.4.3.2 Functions • display_seat_map(showtimeId : string) : Seat • reserve_seat(seatId : string, showtimeId : string) : boolean • release_seat(seatId : string, showtimeId : string) : boolean

3.4.4 Ticket

Ticket
3.4.4.1 Attributes • ticketId : string • showtimeId : string • seatId : string • ticketType : object • price : int • orderId : int • discountType : string
3.4.4.1 Attributes • select_ticket_type(ticketType : object) : object • discount_tickets(discountType : string, price : int) : int • total_ticket_cost(price : int, discountType : string) : int

3.4.5 Orders

Orders
3.4.5.1 Attributes • orderId : int • customerId : string • ticket : ticket (class) • concessions : string • total_cost : double • payment : string • confirmation_number : string • time : int
3.4.5.2 Functions • create_order(customerId : string, ticket : Ticket, concession : string) : Orders • confirm_order(orderId : int, confirmation_number : string) : boolean • cancel_order(orderId : int) : boolean • confirmation(orderId : int) : string

3.4.6 Payment

Payment
3.4.6.1 Attributes • paymentId : unsigned long long • orderId : int • total : double • paymentMethod : string • time : int
3.4.6.2 Functions • record_payment_outcomes(paymentId : unsigned long long, orderId : int, total : double, paymentMethod : string) : boolean • log_payment_failures(paymentId : unsigned long long, orderId : int, paymentMethod : string, time : int) : Logs

3.4.7 Guest User

Guest User
3.4.7.1 Attributes • sessionId : string • email : string • phoneNumber : int
3.4.7.2 Functions • browse_movies(sessionId : string) : Movie • purchase_tickets(sessionId : string, showtimeId : string, seatId : string, ticketType : object) : Orders • submit_feedback(sessionId : string, rating : double, text_msg : string, time : int) : Feedback • pre_order_concessions(sessionId : string, Items : string, total : double) : ConcessionsOrders

3.4.8 Registered User

Registered User
3.4.8.1 Attributes • userId : int • Email : string • phoneNumber : int • password : string • orderHistory : Orders
3.4.8.2 Functions • register(Email : string, phoneNumber : int, password : string) : RegisteredUser • login(Email : string, password : string) : boolean • view_order_history(userId : int) : Orders • browse_movies(sessionId : string) : Movie • purchase_tickets(sessionId : string, showtimeId : string, seatId : string, ticketType : object) : Orders • submit_feedback(sessionId : string, rating : double, text_msg : string, time : int) : Feedback • pre_order_concessions(sessionId : string, Items : string, total : double) : ConcessionsOrders

3.4.9 Administrator

Administrator
3.4.9.1 Attributes • adminId : int • user : string • password : string • accessLevel : int
3.4.9.2 Functions • manage_showtimes(showtimeId: string): void • configure_tickets(ticketType: object, price: double) : void

• security_logs(log: Logs): void • feedback_logs(feedbackId: string): Feedback

3.4.10 Feedback

Feedback
3.4.10.1 Attributes • feedbackId : string • customerId : string • orderId : int • rating : double • text_msg : string • time : int
3.4.10.2 Functions • submit_feedback(customerId: string, orderId: int, rating: double, text_msg: string,): void • view_feedback(feedbackId: string) : Feedback

3.4.11 Concessions Orders

Concessions Orders
3.4.11.1 Attributes • concessionId : int • orderId : int • Items : string • total : double • confirmation : string
3.4.11.2 Functions • browse_menu() : void • add_item(item: string) : void • remove_item(item: string) : void • place_order() : boolean

3.4.12 Logs

Logs
3.4.12.1 Attributes • logId : int • logType: string • user : string • time : int
3.4.12.2 Functions • log_events (logType: string, logId: int, user: string, time: int) : void • review_logs(logType: string): void

3.5 Non-Functional Requirements

Non-functional requirements may exist for the following attributes. Often these requirements must be achieved at a system-wide level rather than at a unit level. State the requirements in the following sections in measurable terms (e.g., 95% of transactions shall be processed in less than a second, system downtime may not exceed 1 minute per day, > 30 day MTBF value, etc).

3.5.1 Performance

NFR-PERF-1 Response Time for Browsing/Search (P1)
The system shall return movie browsing and search results within 2 seconds for at least 95% of requests under normal operating load.

NFR-PERF-2 Seat Map Load Time (P1)
The system shall display the seat map and current seat availability within 3 seconds for at least 95% of requests under normal operating load.

NFR-PERF-3 Checkout Confirmation Time (P1)
For at least 95% of successful purchases, the system shall display an order confirmation within 5 seconds, excluding external payment processor delays.

NFR-PERF-4 High-Demand Degradation (P2)
Under high-demand conditions, the system shall degrade gracefully by applying queueing and/or rate limiting rather than failing, and shall display a user-facing message indicating the user is queued or temporarily limited.

3.5.2 Reliability

NFR-REL-1 No Double-Selling (P1)
The system shall prevent any seat from being sold to more than one customer for the same showtime.

NFR-REL-2 Payment Failure Integrity (P1)
If a payment fails, the system shall not finalize the ticket sale and shall release reserved seats after the reservation timeout.

NFR-REL-3 Daily Error Rate (P2)
The system shall maintain an application-level error rate of less than 1% of total requests per day under normal operating load, excluding failures caused solely by external services.

NFR-REL-4 Scalability Support (P2)
The system shall support horizontal scaling of application services such that adding additional server instances increases total request throughput without requiring changes to the client application.

3.5.3 Availability

NFR-AVAIL-1 Uptime (P1)
The system shall achieve at least 99.9% service availability per calendar month, excluding scheduled maintenance.

NFR-AVAIL-2 Scheduled Maintenance Window (P2)
Scheduled maintenance downtime shall not exceed 1 hour per month and shall be announced to users at least 24 hours in advance.

NFR-AVAIL-3 Load Balancing / Failover (P2)
If an application server instance fails, the system shall continue serving new requests through load balancing and/or failover without a total service outage.

3.5.4 Security

NFR-SEC-1 Secure Transport (P1)
The system shall use HTTPS for all communications between client browsers and the server.

NFR-SEC-2 Payment Data Storage (P1)
The system shall not store raw plain text credit card numbers.

NFR-SEC-3 Login Abuse Protection (P1)
The system shall lock an account or require additional verification after 5 consecutive failed login attempts within 10 minutes.

NFR-SEC-4 Session Timeout (P2)
The system shall expire inactive user sessions after a configurable duration (default 30 minutes).

NFR-SEC-5 Admin Access Control and Logging (P1)
Administrative functions shall be accessible only to authorized administrators, and unauthorized admin access attempts shall be denied and logged.

NFR-SEC-6 Service/Database Separation (P2)
The system shall restrict direct public internet access to databases such that databases are accessible only from authorized internal services.

3.5.5 Maintainability

NFR-MAINT-1 Modular Components
The system will be organized into modular components such that changes to one module will not require changes to unrelated modules.

NFR-MAINT-2 Verbose Diagnostic Logging
The system will record server side logs for ticket purchases, failed payments, seat-hold timeouts, bot prevention events, administrator actions and include a timestamp and relevant identifier to support troubleshooting.

NFR-MAINT-3 Configurable Operational Parameters
The system will allow changes to operational parameters through configuration without requiring code changes.

NFR-MAINT-4 Maintainable Deployment Updates
The system will support updating a single containerized service without requiring downtime of the entire system, except during scheduled maintenance windows.

3.5.6 Portability

NFR-PORT-1 Browser Compatibility (P1)
The customer-facing system shall operate on modern web browsers (Chrome, Firefox, Safari, Edge) without requiring installation of a mobile app.

NFR-PORT-2 Device Compatibility (P2)
The system shall function on common device types capable of running a modern browser, including desktop, laptop, and tablet devices.

NFR-PORT-3 Server Deployment Portability (P3)
The system's server-side services shall be deployable using containerization such that each major service can run in a container without requiring code changes across environments.

NFR-PORT-4 Environment Portability (P3)
The containerized services shall be deployable on common Linux-based server environments without requiring operating-system-specific features.

3.6 System Description

The Movie Theater Web Ticketing System (MTWTS) is a browser-based application that enables customers to browse available movies, view showtimes and seat maps, select tickets (including discount categories), and complete purchases through an external payment processor. The system also supports guest checkout, registered-user accounts (login, order history), concession

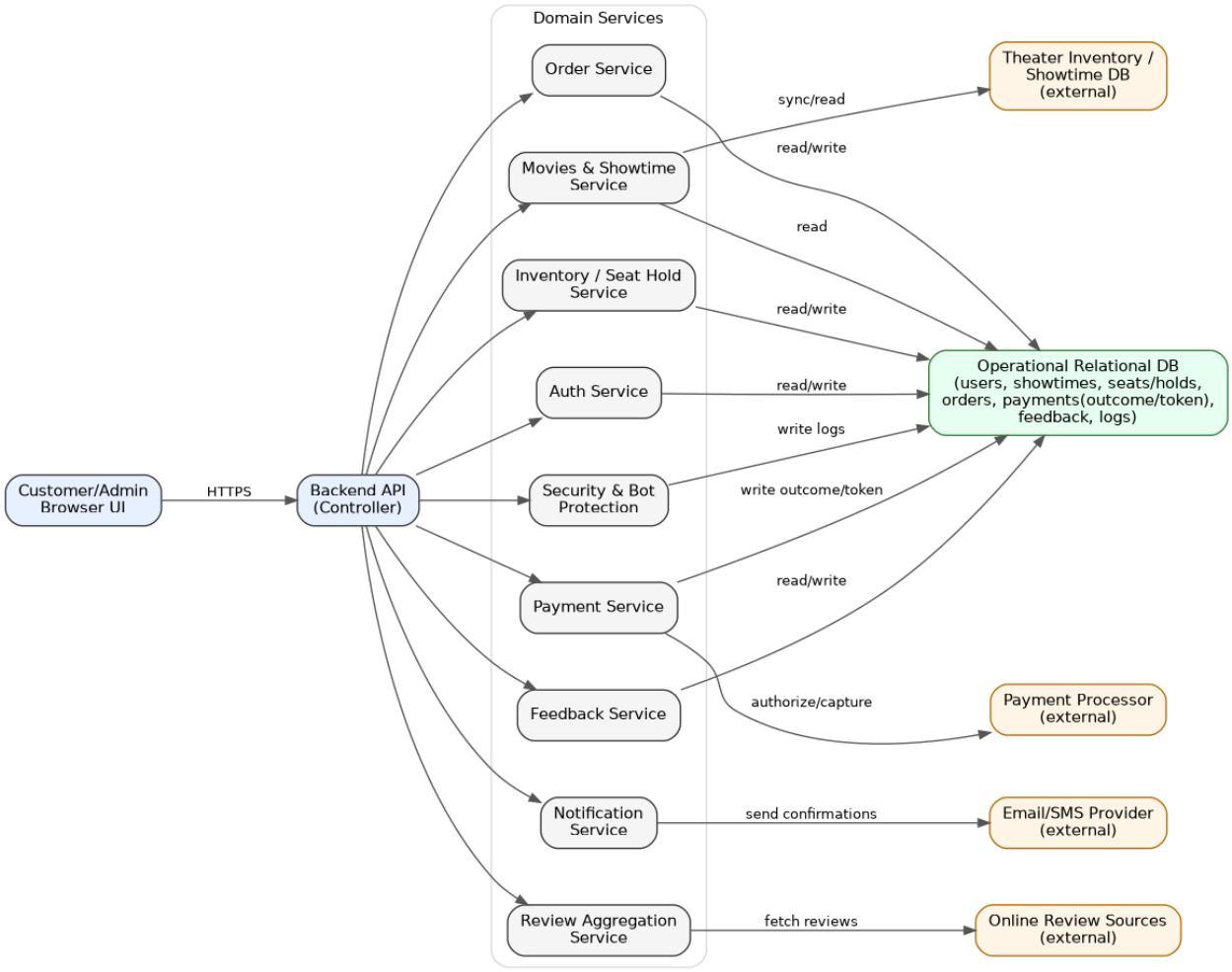
pre-orders, customer feedback submission, and an administrator portal to manage showtimes, ticket pricing/discount rules, and to review security/bot activity logs.

At a high level, the system consists of:

- Web Client UI used by customers and administrators.
- Backend API that enforces business rules (purchase windows, ticket limits, seat holds, authentication/authorization).
- Domain services for inventory/seat holds, ticket ordering, payment processing, reviews retrieval, feedback management, and security/bot protection.
- A database for persistent storage of movies/showtimes, seat inventory state, users, orders, payments (no raw card data), feedback, and logs.
- External integrations for showtime/ticket inventory data, payment processing, online reviews, and email/SMS notifications.

3.7 Software Architecture Overview

3.7.1 Software Architecture (SWA) Diagram



Design 2.0 of Software Architecture Diagram that includes single operational DB and external data sources.

3.7.2 SWA Description (Components + Connectors)

Client Layer

- Customer Browser UI: Provides pages for browsing, showtime viewing, seat selection, checkout, order confirmation/history, and feedback submission.
- Admin Browser UI: Provides secure access to admin-only features such as showtime management, pricing/discount configuration, and log review.

Backend API / Controller Layer

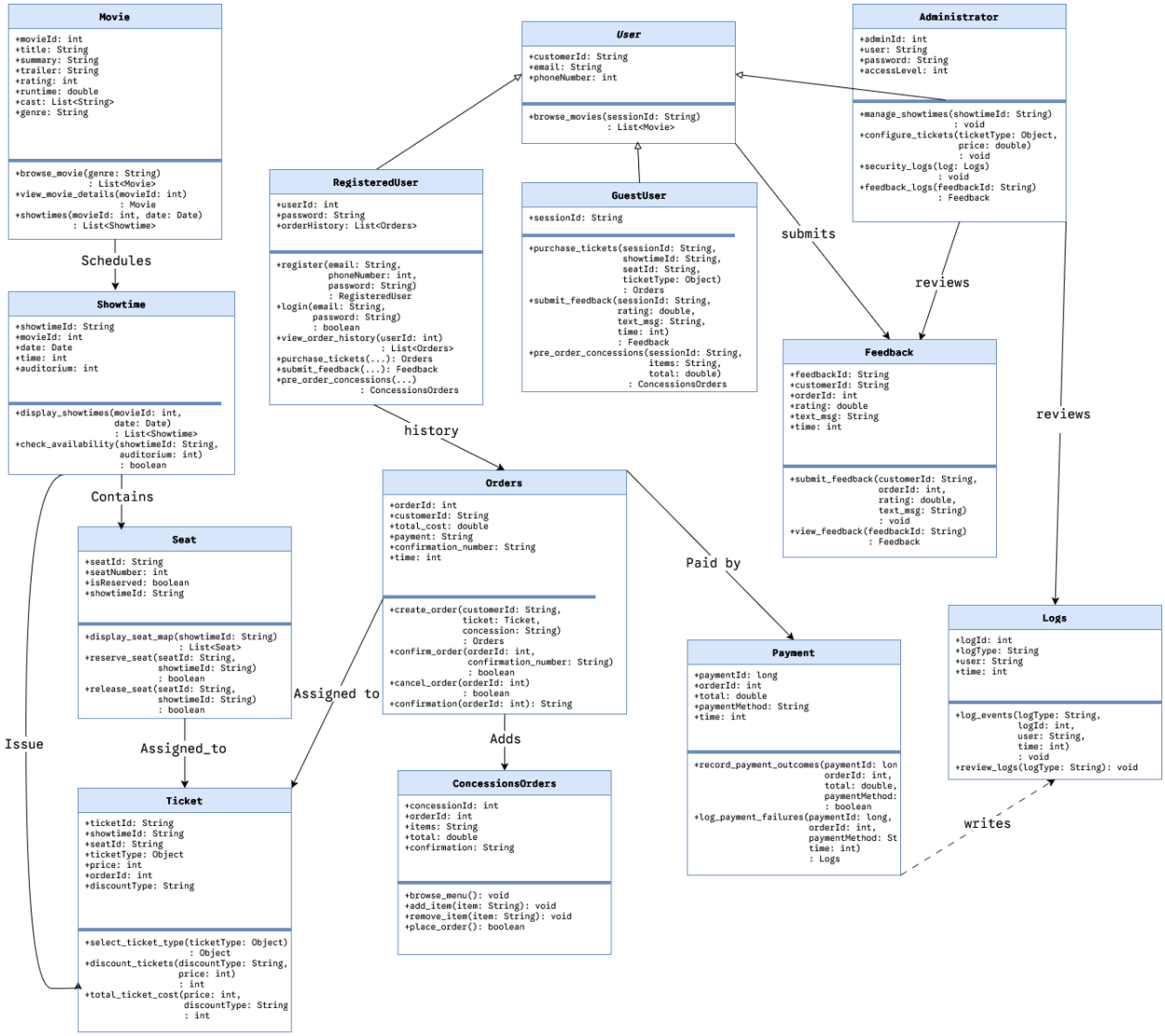
- Exposes REST-style endpoints (or equivalent) for all features.
- Performs request validation, session handling, authentication checks, and delegates domain logic to services.
- Primary connector: HTTPS between browsers and backend.

Domain Services

- Auth Service

    - Handles registration, login, session tokens, and authorization checks (admin vs. customer).
- Movies & Showtime Service
    - Retrieves movies/showtimes and movie details; coordinates caching where needed.
- Inventory / Seat Hold Service
    - Manages real-time seat availability, temporary holds (e.g., 5 minutes), final reservation commits upon successful payment.
- Order Service
    - Creates and manages ticket orders and concession orders; computes totals and generates confirmation numbers.
- Payment Service
    - Integrates with external payment processor; stores only payment outcomes/tokens.
- Review Aggregation Service
    - Retrieves critic quotes/review snippets from online sources; handles fallback messaging when unavailable.
- Feedback Service
    - Stores and retrieves customer feedback; enables admin review.
- Security & Bot Protection Service
    - Rate limits suspicious activity, triggers CAPTCHA/human verification, and logs security events.
- Notification Service
    - Sends email/SMS confirmations, verification codes, and other user notifications.
- Data Layer
    - Relational Database
        - Stores persistent business entities: users, movies/showtimes, seats/holds, orders, payments (token/outcome), feedback, and logs.
        - Connectors: services communicate with DB via a secure internal connector (ORM/SQL driver), not exposed publicly.
- External Integrations
    - Theater Inventory/Showtime DB: source of truth for showtime and seat availability synchronization.
    - Payment Processor: Payment authorization/capture, returns transaction token/outcome.
    - Online Review Sources: critic quote and review data.
    - Email/SMS Provider: confirmations and verification messages.

3.7.3 UML Class Diagram



3.7.4 UML Class Descriptions (Attributes + Operations)

Class: Movie

Represents a movie title and its static metadata. Stores all descriptive information about a film and provides methods for browsing and retrieving showtime listings.

Attributes:

- **movieId: int** - unique identifier for the movie
- **title: String** - full display title
- **summary: String** - short plot description
- **trailer: String** - URL or embed key for the trailer video
- **rating: int** - content rating (e.g., PG, PG-13, R encoded as int)

- **runtime: double** - length of the film in minutes
- **cast: List<String>** - list of featured cast member names

genre: String - movie genre e.g. "Action", "Comedy"

Operations:

- **browse_movie(genre: String): List<Movie>** - returns movies filtered by genre
- **view_movie_details(movieId: int): Movie** - returns full detail object for a selected movie
- **showtimes(movieId: int, date: Date): List<Showtime>** - returns all showtimes for the movie on a given date

Class: Showtime

Represents a scheduled screening of a movie at a specific date, time, and auditorium. Seat inventory is scoped to each Showtime instance to prevent double-selling across different screenings.

Attributes:

- **showtimeId: String** - unique identifier for this screening
- **movieId: int** - foreign-key reference to the Movie being shown
- **date: Date** - calendar date of the screening
- **time: int** - start time (implementation may use a DateTime type)
- **auditorium: int** - auditorium or screen number within the theater

Operations:

- **display_showtimes(movieId: int, date: Date): List<Showtime>** - returns available showtimes for a movie on a specific date
- **check_availability(showtimeId: String, auditorium: int): boolean** - returns true if at least one seat remains available for the showtime

Class: Seat

Represents a single seat instance scoped to a specific Showtime. Tracking availability per-showtime ensures the same physical seat can be sold for different screenings without conflict.

Attributes:

- **seatId: String** - unique identifier for this seat record
- **seatNumber: int** - display number of the seat within the auditorium
- **isReserved: boolean** - true if the seat is currently held or sold
- **showtimeId: String** - reference to the owning Showtime

Operations:

- **display_seat_map(showtimeId: String): List<Seat>** - returns the full seat map with availability status for rendering in the UI
- **reserve_seat(seatId: String, showtimeId: String): boolean** - places a temporary hold on the seat; returns true on success

- **release_seat(seatId: String, showtimeId: String): boolean** - releases a held seat on cancellation or reservation timeout

Class: Ticket

Represents a purchasable ticket tied to a specific Seat and Showtime. Handles pricing rules for discount categories (adult, child, senior, student, etc.) and computes the final cost before checkout.

Attributes:

- **ticketId: String** - unique ticket identifier
- **showtimeId: String** - reference to the target Showtime
- **seatId: String** - reference to the assigned Seat
- **ticketType: Object** - category object e.g Adult / Child / Senior / Student
- **price: int** - base ticket price in cents
- **orderId: int** - reference to the containing Order
- **discountType: String** - discount category label applied to this ticket

Operations:

- **select_ticket_type(ticketType: Object): Object** - sets the ticket category and returns the updated type object
- **discount_tickets(discountType: String, price: int): int** - applies the discount rule and returns the adjusted price
- **total_ticket_cost(price: int, discountType: String): int** - computes the final cost for this ticket including all applicable discounts

Class: Orders

Represents a checkout order aggregating one or more Tickets and optional ConcessionsOrders. This is the primary entity that users confirm, track, and cancel. Holds the confirmation number issued after successful payment.

Attributes:

- **orderId: int** - unique order identifier
- **customerId: String** - identifier of the customer who placed the order
- **total_cost: double** - total amount charged including taxes and fees
- **payment: String** - payment status or token summary (no raw card data stored)
- **confirmation_number: String** - human-readable confirmation code for the customer
- **time: int** - creation timestamp of the order

Operations:

- **create_order(customerId: String, ticket: Ticket, concession: String): Orders** - initializes a new pending order object

- **confirm_order(orderId: int, confirmation_number: String): boolean** - finalizes the order after successful payment processing
- **cancel_order(orderId: int): boolean** - cancels the order and triggers seat release for all associated tickets
- **confirmation(orderId: int): String** - returns the confirmation string for display in the UI or email receipt

Class: Payment

Handles recording of payment outcomes and logging of failures. Integrates with the external payment processor. Stores only transaction outcomes and tokens (raw card numbers are never stored).

Attributes:

- **paymentId: long** - unique payment transaction identifier
- **orderId: int** - reference to the associated Order
- **total: double** - total amount of the transaction
- **paymentMethod: String** - method used such as "VISA", "MASTERCARD", "PAYPAL" and more.
- **time: int** - timestamp of the payment event

Operations:

- **record_payment_outcomes(paymentId: long, orderId: int, total: double, paymentMethod: String): boolean** - persists the transaction result or token; returns true on success
- **log_payment_failures(paymentId: long, orderId: int, paymentMethod: String, time: int): Logs** - creates a Logs entry capturing details of a failed payment event

Class: User (abstract)

Abstract base class providing shared identity and contact attributes for all user types. GuestUser and RegisteredUser both inherit from User. This class is not instantiated directly.

Attributes:

- **customerId: String** - session or account identifier for this user
- **email: String** - contact email address
- **phoneNumber: int** - contact phone number

Operations:

- **browse_movies(sessionId: String): List<Movie>** - returns the current list of available movies to display on the home page

Class: GuestUser (extends User)

Session-based user who can browse movies, purchase tickets, submit feedback, and pre-order concessions without creating a persistent account. All state is tied to the session and expires on browser close or timeout.

Attributes:

- **sessionId: String** - ephemeral session token for tracking the guest during their visit

Operations:

- **purchase_tickets(sessionId: String, showtimeId: String, seatId: String, ticketType: Object): Orders** - creates an order for the selected showtime, seat, and ticket type
- **submit_feedback(sessionId: String, rating: double, text_msg: String, time: int): Feedback** - submits a feedback entry linked to the current guest session
- **pre_order_concessions(sessionId: String, items: String, total: double): ConcessionsOrders** - places a concession pre-order associated with the current showtime

Class: RegisteredUser (extends User)

Persistent account user with stored credentials and order history. Inherits all GuestUser capabilities and additionally supports secure login, account registration, and retrieval of past orders.

Attributes:

- **userId: int** - unique account identifier
- **password: String** - hashed password credential (never stored in plaintext)
- **orderHistory: List<Orders>** - list of all past and current orders for this account

Operations:

- **register(email: String, phoneNumber: int, password: String): RegisteredUser** - creates a new account and returns the new RegisteredUser object
- **login(email: String, password: String): boolean** - validates credentials and returns true on successful authentication
- **view_order_history(userId: int): List<Orders>** - returns the complete order history for the account
- **purchase_tickets(...): Orders** - same as GuestUser but linked to the persistent account
- **submit_feedback(...): Feedback** - same as GuestUser but linked to the account ID
- **pre_order_concessions(...): ConcessionsOrders** - same as GuestUser but linked to the account ID

Class: Administrator

Privileged user role for theater operations staff. Manages showtimes, ticket types and pricing, and reviews system logs and customer feedback. Accessed through a separate, restricted admin login endpoint.

Attributes:

- **adminId: int** - unique administrator identifier

- **user: String** - admin username or account name
- **password: String** - hashed admin password
- **accessLevel: int** - permission tier e.g 1 = standard, 2 = sudo-admin

Operations:

- **manage_showtimes(showtimeId: String): void** - create, update, or remove a showtime record in the system
- **configure_tickets(ticketType: Object, price: double): void** - set pricing rules and discount configurations for ticket categories
- **security_logs(log: Logs): void** - retrieve and review security and bot-detection log entries
- **feedback_logs(feedbackId: String): Feedback** - retrieve and review specific customer feedback submissions

Class: Feedback

Stores customer feedback submissions with an optional reference to a specific order. Supports both anonymous guest and authenticated user feedback. Administrators can retrieve and review all entries.

Attributes:

- **feedbackId: String** - unique feedback entry identifier
- **customerId: String** - submitting user identifier (session ID or account ID)
- **orderId: int** - optional reference to an associated order
- **rating: double** - numeric rating value on a 1.0 to 5.0 scale
- **text_msg: String** - freeform written feedback from the customer
- **time: int** - timestamp when the feedback was submitted

Operations:

- **submit_feedback(customerId: String, orderId: int, rating: double, text_msg: String): void** - validates and persists a new feedback entry
- **view_feedback(feedbackId: String): Feedback** - retrieves a specific feedback record by its ID

Class: ConcessionsOrders

Represents concession items pre-ordered by a customer for a specific showtime. Linked to an Orders record. The customer receives a pickup confirmation code upon successful payment.

Attributes:

- **concessionId: int** - unique concession order identifier
- **orderId: int** - reference to the parent ticket order
- **items: String** - serialized list of selected menu items and quantities
- **total: double** - total cost of the concession order
- **confirmation: String** - pickup confirmation code issued after payment

Operations:

- **browse_menu(): void** - loads and displays the available concession menu items
- **add_item(item: String): void** - adds a concession item to the current order
- **remove_item(item: String): void** - removes a concession item from the current order
- **place_order(): boolean** - submits the concession order for payment processing; returns true on success

Class: Logs

Tracks security, administrative, payment, and bot-detection events for audit and operational review. Log entries are append-only and include the user/session involved, the event type, and a timestamp.

Attributes:

- **logId: int** - unique log entry identifier
- **logType: String** - event category - e.g "AUTH", "BOT", "PAYMENT", "ADMIN"
- **user: String** - user or session identifier associated with the event
- **time: int** - timestamp when the event was recorded

Operations:

- **log_events(logType: String, logId: int, user: String, time: int): void** - creates a new log entry for a system event
- **review_logs(logType: String): void** - retrieves and displays log entries filtered by type for administrator review

3.7.5 Development Plan and Timeline

Task Partitioning (Major Work Streams)

1. Frontend UI
    a. Customer UI: movie browsing, showtimes, seat selection, checkout, confirmation, feedback.
    b. Admin UI: manage showtimes/prices/discounts; view logs/feedback.
2. Backend API + Business Logic
    a. Authentication/authorization, purchase window rules,
3. Data Layer
    a. Schema design, DB access layer, data integrity constraints, migrations/seed data.
4. Integrations:
    a. Payment processor integration, email/SMS notifications, review sources integration.
5. Security/Monitoring
    a. Rate limiting/bot detection triggers, audit logging, admin access logging.
6. Testing + Documentation
    a. Unit tests, integration tests, API docs, deployment notes.

Team Responsibilities (Group #5)

- Felipe Muñoz
    - Backend API (Order + Inventory/Seat Holds), integration testing.
- Miriam Valenzuela
    - Frontend UI implementation, usability/accessibility checks, UI testing.
- Victor Lopez
    - Database schema + persistence layer, logging/monitoring features, admin tools.

Estimated Timeline (4-Week Plan)

- Week 1: Architecture finalized; DB schema + core API scaffolding; UI wireframes.
- Week 2: Movie/showtime browsing + seat map; seat holds; basic checkout flow.
- Week 3: Payment integration + confirmations; admin panel basics; feedback system.
- Week 4: Bot protection + logging; hardening; testing; final documentation & polish.

3.8 Logical Database Requirements

Types of information used by various functions;

- *User Accounts*
- *Movies & Showtimes*
- *Seats*
- *Orders*
- *Tickets*
- *Logs*
- *Pricing & Discounts*

Frequency of use;

- *Seat Availability(used every time that someone browses a showtime)*
- *User Authentication (used every time that someone signs in)*
- *Orders & Tickets (used every time someone completes an order)*
- *Movies & Showtimes (Changed Frequently by customers & changed moderately by admins)*
- *Logs(Gets written to more frequently as it is based on other functions , also read occasionally by admins)*
- *Pricing & Discounts(Not updated as frequently , only during sales or other special events)*

Accessing capabilities;

- *Customers (Read movies / Showtimes , Read & Write Orders / Tickets)*
- *Admins (Read / Write on all information)*
- *Backend (Inventory Service & Order Service)*

Data entities and their relationships;

- *User (Can have multiple Orders)*
- *Order(Can have multiple Tickets)*
- *ShowTime(Connected to one Auditorium & One Movie)*
- *Seat(Connected to one Auditorium & One Showtime)*
- *Movie(Can have multiple Showtimes)*

Integrity constraints;

- *Two Tickets cannot have the same seat for same Showtime*
- *Seat cannot be held by more than one customer at the same time*
- *Enforcing transaction limits*
- *Payment Completed before an Order is Confirmed*
- *Logs cant be deleted or changed*
- *Unique Id's for all records*

Data retention requirements.

- *Logs maintained for minimum of 6 months for security*
- *Seat holds released after 5 mins*
- *Order & Ticket records kept for minimum 1 year*
- *User Information retained until deleted*
- *Raw Payment Information never stored*

3.9 Other Requirements

OR-1 Notifications and Receipts (P2)

- **OR-1.1** The system shall display an on-screen order confirmation after a successful ticket purchase or concession pre-order. (P2)

- **OR-1.2** The system shall send an order confirmation/receipt to the customer via email and/or SMS when a contact method is provided. (P2)

- **OR-1.3** The confirmation message shall include: confirmation number, movie title, showtime date/time, auditorium (if applicable), seats purchased, ticket types, total cost, and pickup instructions for concessions (if applicable). (P2)

OR-2 Refunds, Cancellations, and Reversals (P2)

- **OR-2.1** If a confirmed order is cancelled (by user or admin), the system shall release the associated seats back to "Available" inventory. (P2)

- **OR-2.2** If a payment reversal/refund is supported, the system shall record the refund status and timestamp and link it to the original order record. (P3)

OR-3 Concession Pickup Verification (P3)

- **OR-3.1** For concession pre-orders, the system shall generate a pickup confirmation code shown to the customer on-screen and included in the receipt. (P3)

- **OR-3.2** The system shall allow theater staff (admin role) to mark a concession order as "Picked Up" using the confirmation code. (P3)

OR-4 Business Rule Configuration (P2)

- **OR-4.1** The system shall allow administrators to configure operational parameters through configuration (without code changes), including: seat-hold timeout duration, failed-login lockout thresholds, and verification/rate-limit thresholds. (P2)

OR-5 Data Privacy and Minimal Collection (P2)

- **OR-5.1** The system shall collect only the minimum customer information required to complete purchases and deliver confirmations (e.g., email or phone if notifications are requested). (P2)

- **OR-5.2** The system shall provide a way for registered users to request deletion of their account data, subject to retention requirements for orders/logs. (P3)

OR-6 Auditability and Traceability (P2)

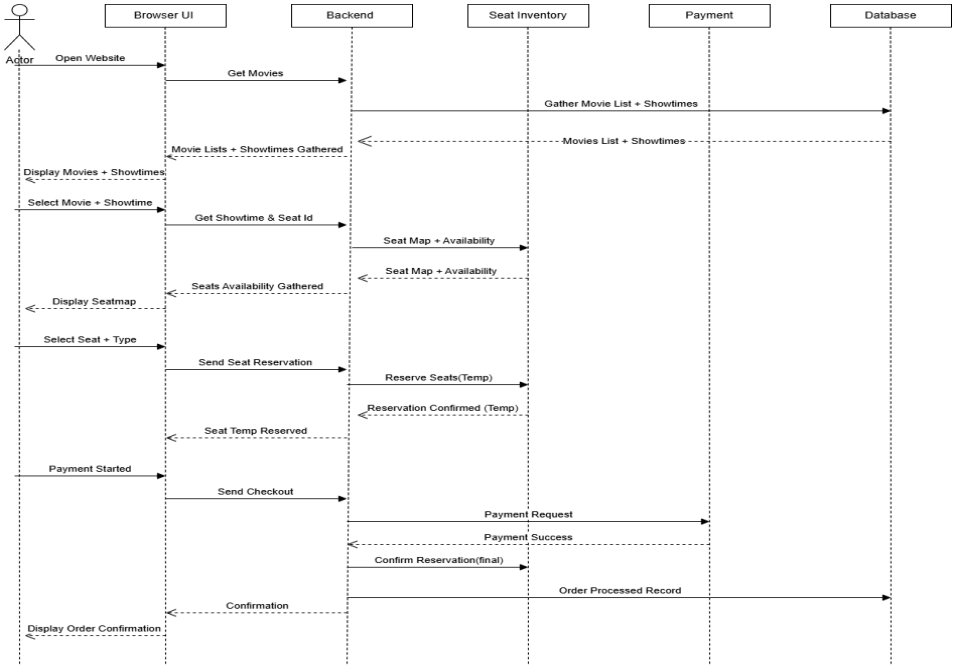
- **OR-6.1** The system shall create audit log entries for security and administrative actions, including timestamp and relevant identifier. (P2)

- **OR-6.2** The system shall not allow users (including admins) to modify or delete security logs through the UI. (P2)

4. Analysis Models

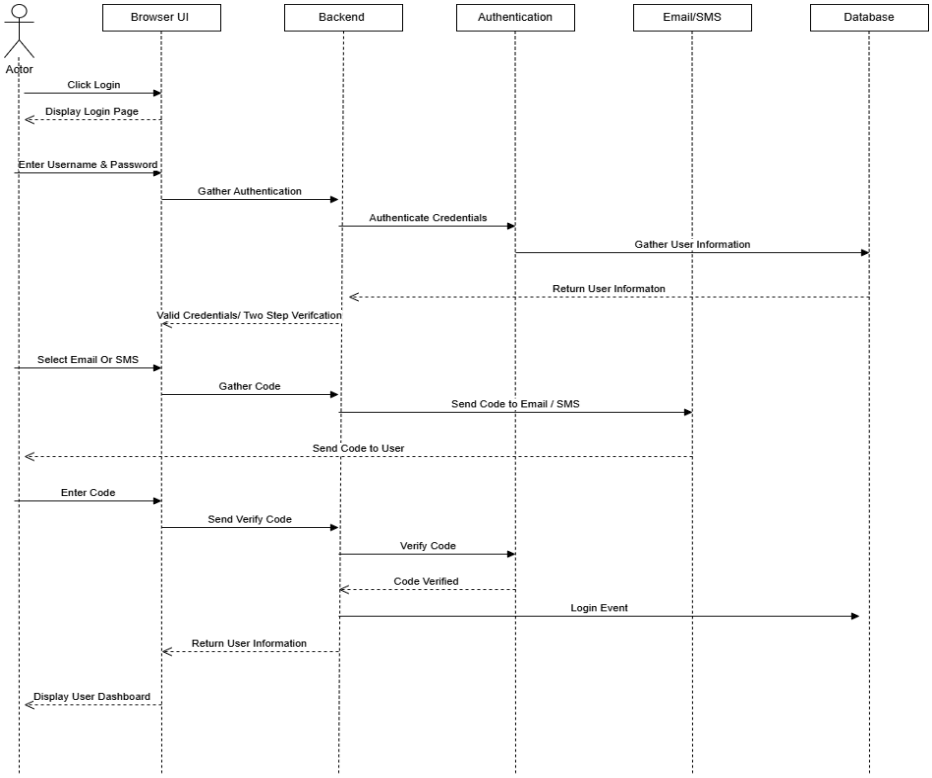
4.1 Sequence Diagrams

Buying Movie Ticket Sequence Diagram:
This sequence diagram shows the interaction between the customer and the MTWTS while purchasing tickets.



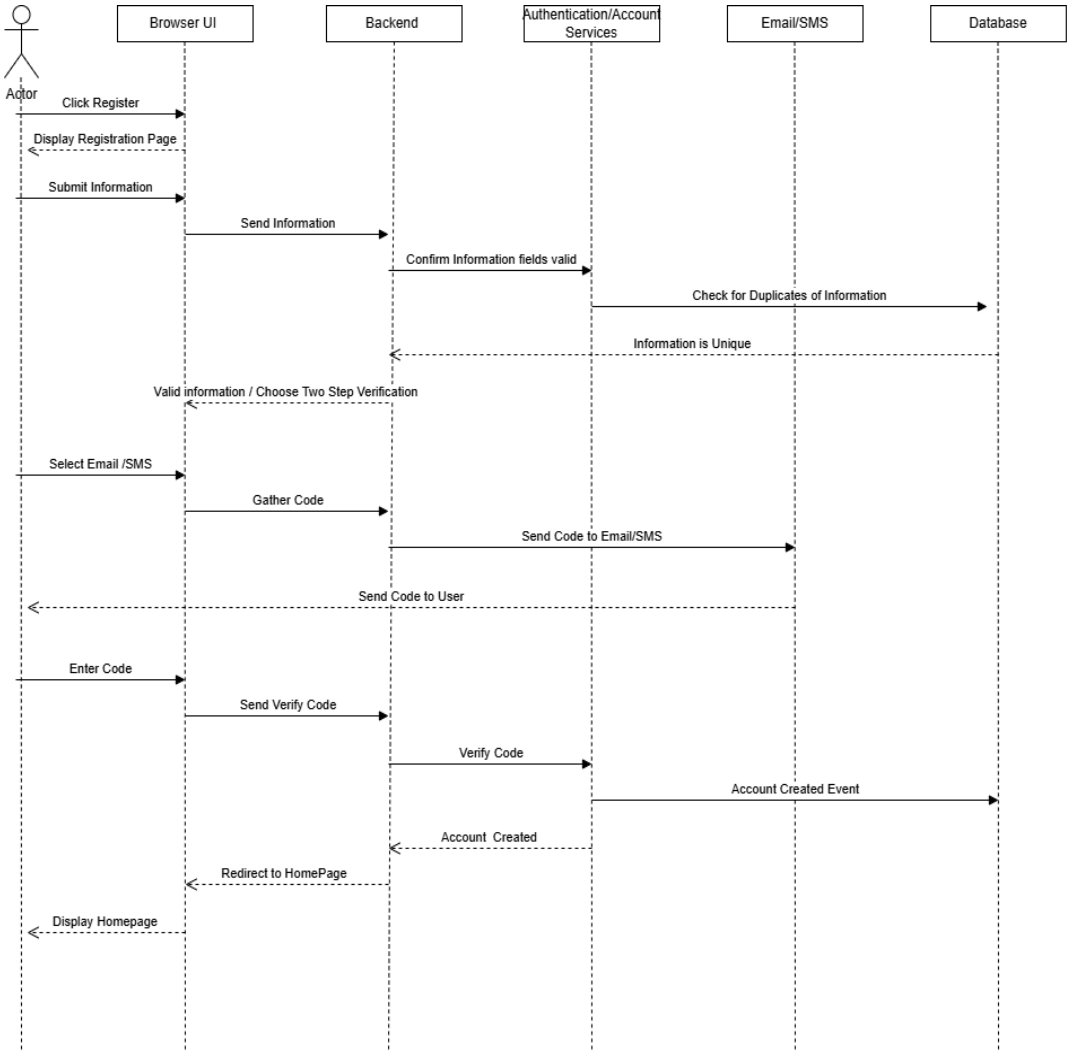
The sequence begins when the customer opens the browser and is browsing through the movies, which gets sent to the Backend , where the backend then gathers the current Movies and Showtimes from the Movie Theater Database. These are then gathered back and displayed to the User through the UI. When a customer selects a Movie & Showtime the backend then gathers seat availability from the Seat Inventory forwards that back to the UI to display the available seats in a Seatmap in the UI. Once a seat is selected then a hold is put on the seat through the backend and seat inventory until payment is processed. Once payment is started there is a payment confirmation , then a reservation confirmation to which the process is done and completed.

Login For Registered User Sequence Diagram:
This sequence diagram shows the process of a registered user and the MTWTS during a login process.



This process starts with the user getting into the Login Page. Once they have entered their information , it will then be sent to the backend to verify with an authentication to ensure correct credentials. Then the credentials are fetched from the database and returned back to the backend. Before the credentials are displayed ,the backend asks for Two Step Verification , which then the User can confirm with either Email or SMS. The backend then creates a code that is sent to either Email / SMS . After the user is then prompted to enter the code which the backend will verify is correct. Once the correct code is verified the event is logged and the User is then displayed with their Dashboard.

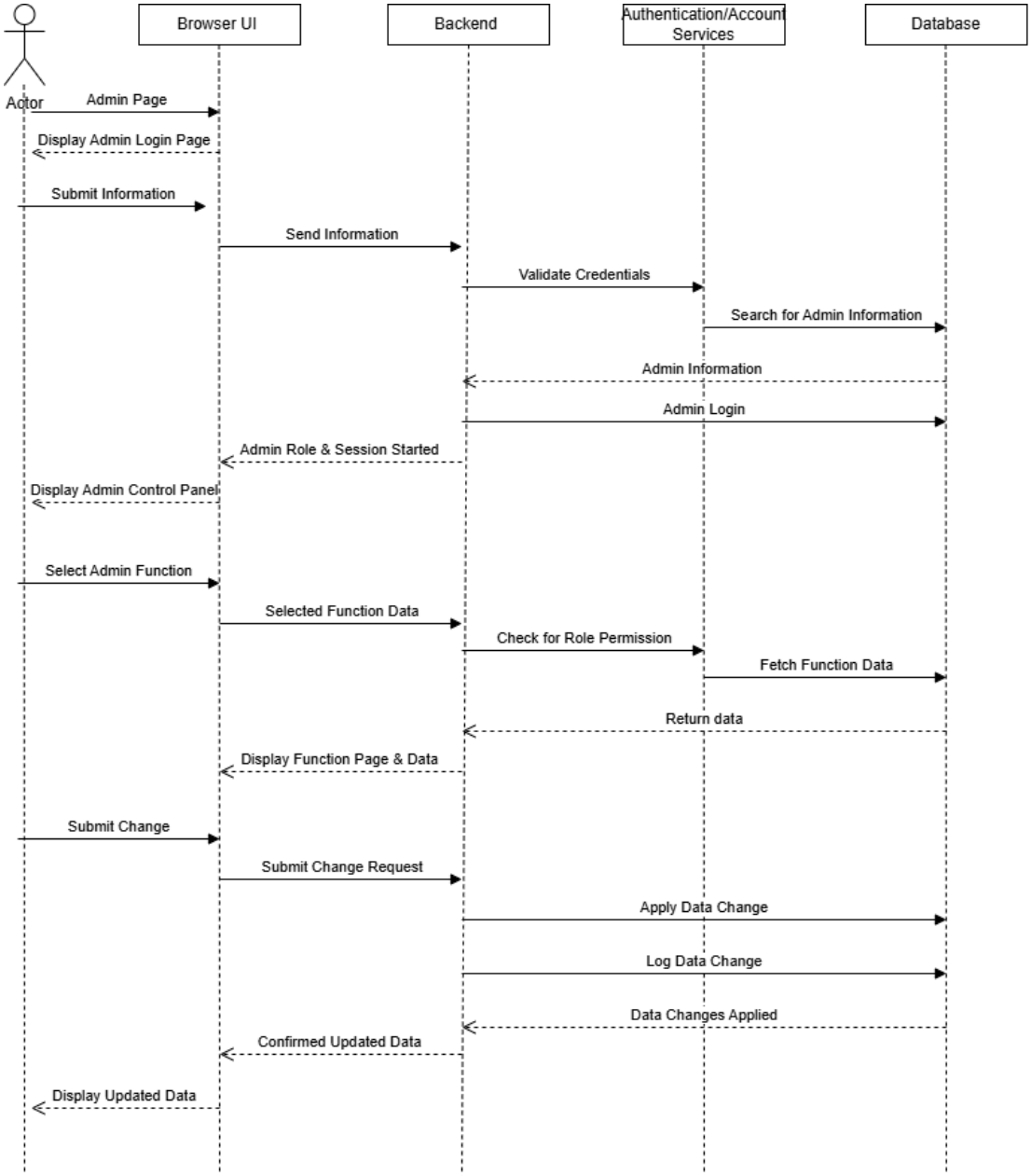
Register An Account Sequence Diagram:
This Sequence Diagram shows the interaction between a guest user and the MTWTS while in the account registration process.



The process starts with the user getting to the registration page. They then submit their information, the backend then confirms that the information in both fields are valid and double checks with the database that nothing is a duplicate. Once information is confirmed to be unique the user is then prompted by the backend to choose a method for their Two Step Verification. Once the user selects their preferred method, the backend creates a code, sends the code to the Email/SMS services , which then sends the code to the user. The User is then prompted to enter in the code and the backend verifies this code matches and the Account Created Event processed by the database. The account is then created and the User is redirected to the Home page.

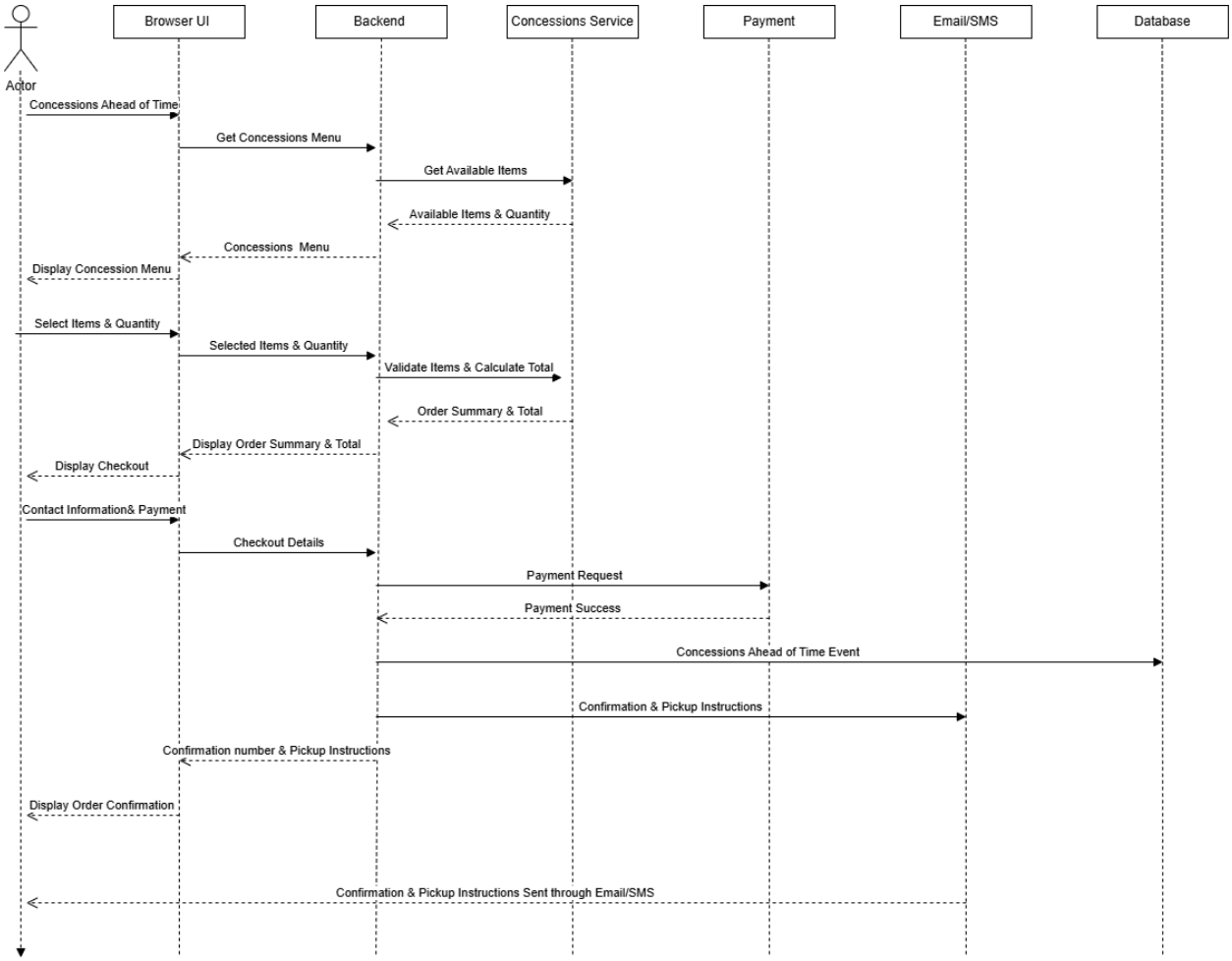
Administration Control Panel Sequence Diagram:

This Sequence Diagram shows the interaction with the Administration Control Panel and the MTWTS from logging in, to performing a change in the system.



This process starts once the user is displayed on the Admin page. The user then submits their information which the backend then validates the credentials with the authentication and searches for the admin information. The admin information is then sent back from the database to the backend , and an Admin login Event is Logged. The user is then Displayed the Admin Page. Once the admin wants to change something first ,the backend checks if the role permission that the admin has is valid to do the change, and it does that by checking with the backend n once checked the database returns the data and the backend sends it to be displayed. Once an admin wants to submit a change it is sent to the backend , then the change is applied to the database, logged, and applied. Confirmed by the backend and then ultimately displayed back as updated data.

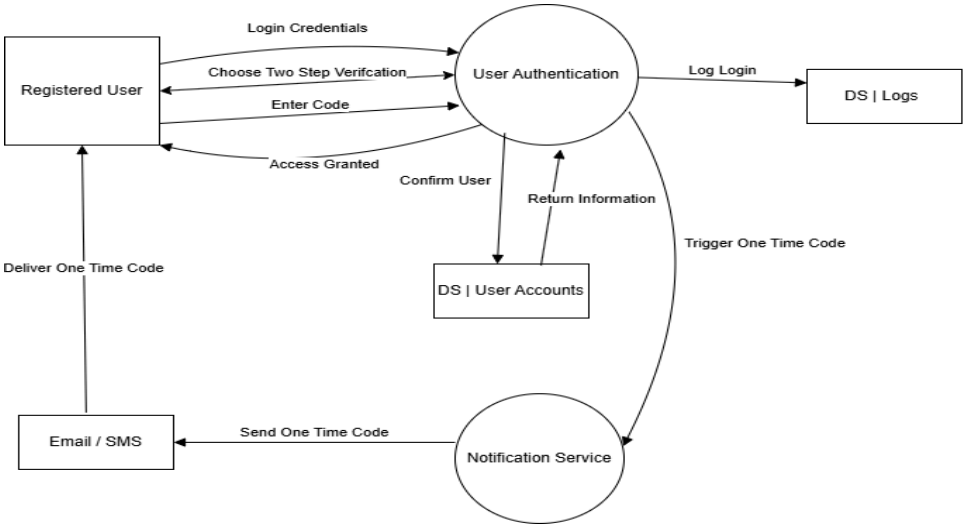
Purchase Food Ahead of Time Sequence Diagram:
This sequence diagram shows the process of purchasing concession stand food within the MTWTS before a showing.



This process starts with the user going to the Concession Pre-Order tab and browsing the menu that the backend retrieves from the concessions service which has all available items. These items are then displayed back to the user. The user then selects their items and quantity and the backend double checks with the concession service that the items are valid and calculates the total. The Order Summary is then displayed for the user . The user is then prompted to enter their contact & payment information. The payment information is then sent to the payment services that return with a payment success and then a Concession Pre Order Event is logged. Confirmation & pickup instructions are then sent to the user via the UI and via Email / SMS

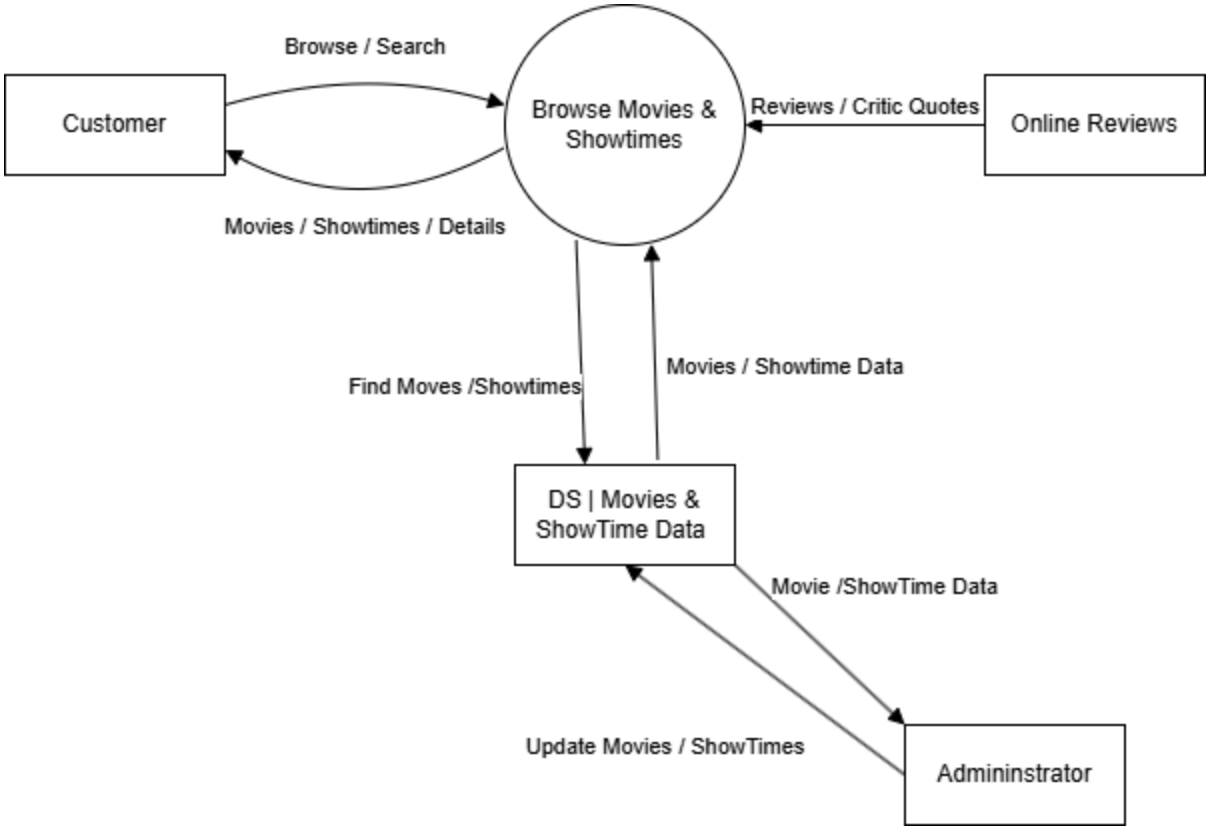
4.3 Data Flow Diagrams (DFD)

User Authentication Data Flow Diagram:
This Data Flow Diagram shows how data moves through the MTWTS during the user authentication process.



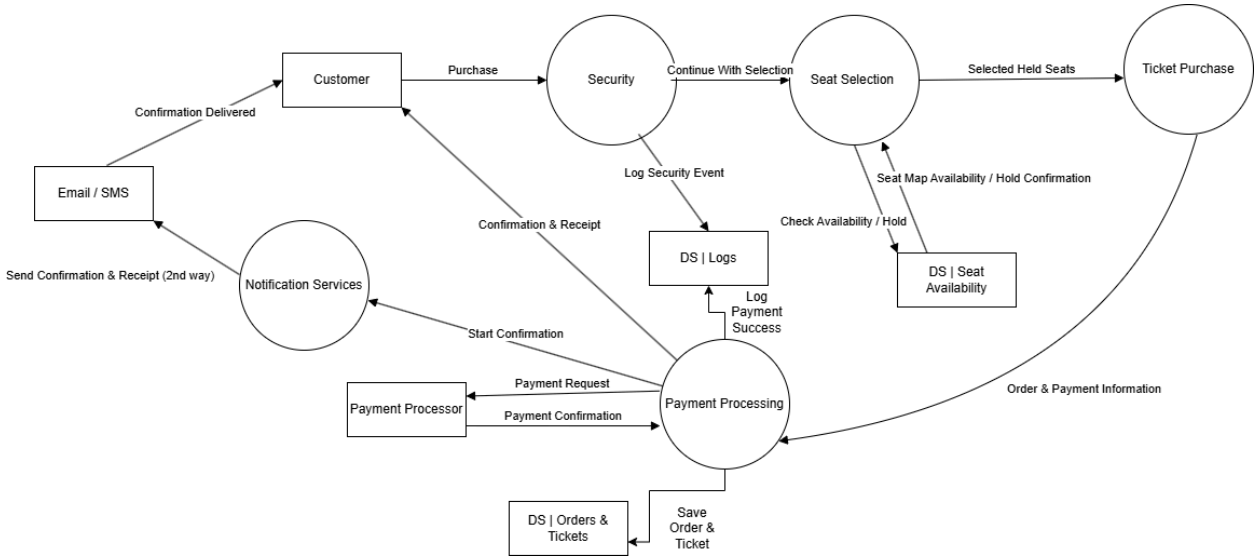
In the Data flow shown here we can see the flow of the credentials as a user authenticates. The user submits their login credentials to the User Authentication process , which confirms their identity with the Data store User Accounts and returns the relevant information. Once successful the system logs the event. Then the Notification Services are triggered and Two Step verification is begun with the user submitting a code to get granted access.

Movie Browsing & Showtimes Data Flow Diagram:
This Data Flow Diagram shows how data moves through the MTWTS during the movie browsing process.



The customer sends a browse request to the browse process and then receives the retrievals of Movies and their Showtimes along with details about the Movie. The browse Movies process then also gathers from online reviews and displays it back to the user, along with retrieving information from the Movie Theater Database. The administrator can directly interact with the Movies and Showtimes Data therefore affecting what the user will see in the end.

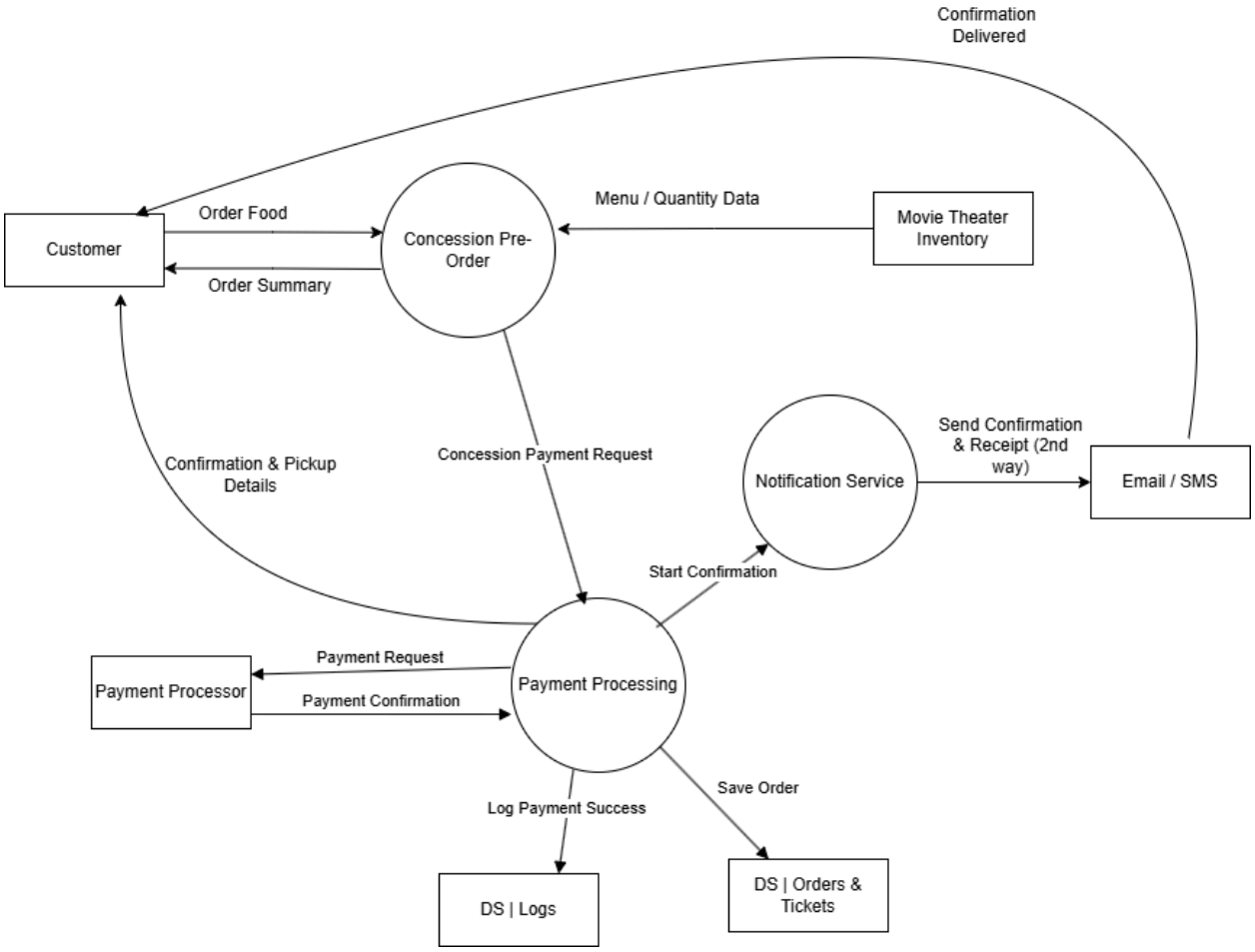
Seat Selection & Ticket Purchase Data Flow Diagram:
This Data Flow Diagram shows how data moves through the MTWTS during the seat selection and ticket purchase process.



The customer initiates the process with a purchase request which is then checked for security purposes to make sure there is no suspicious activity and logs the event. The seat selection process then checks the DS Seat Availability to verify which of the seats are available and places a temporary hold on the seats. Then the seats are put through the Ticket purchase process and taken to the payment information page. The payment process then interacts with the payment processor and saves the order as well as displaying a confirmation receipt. This process then also sends another confirmation . but to a notification service where the customer can get through Email / SMS.

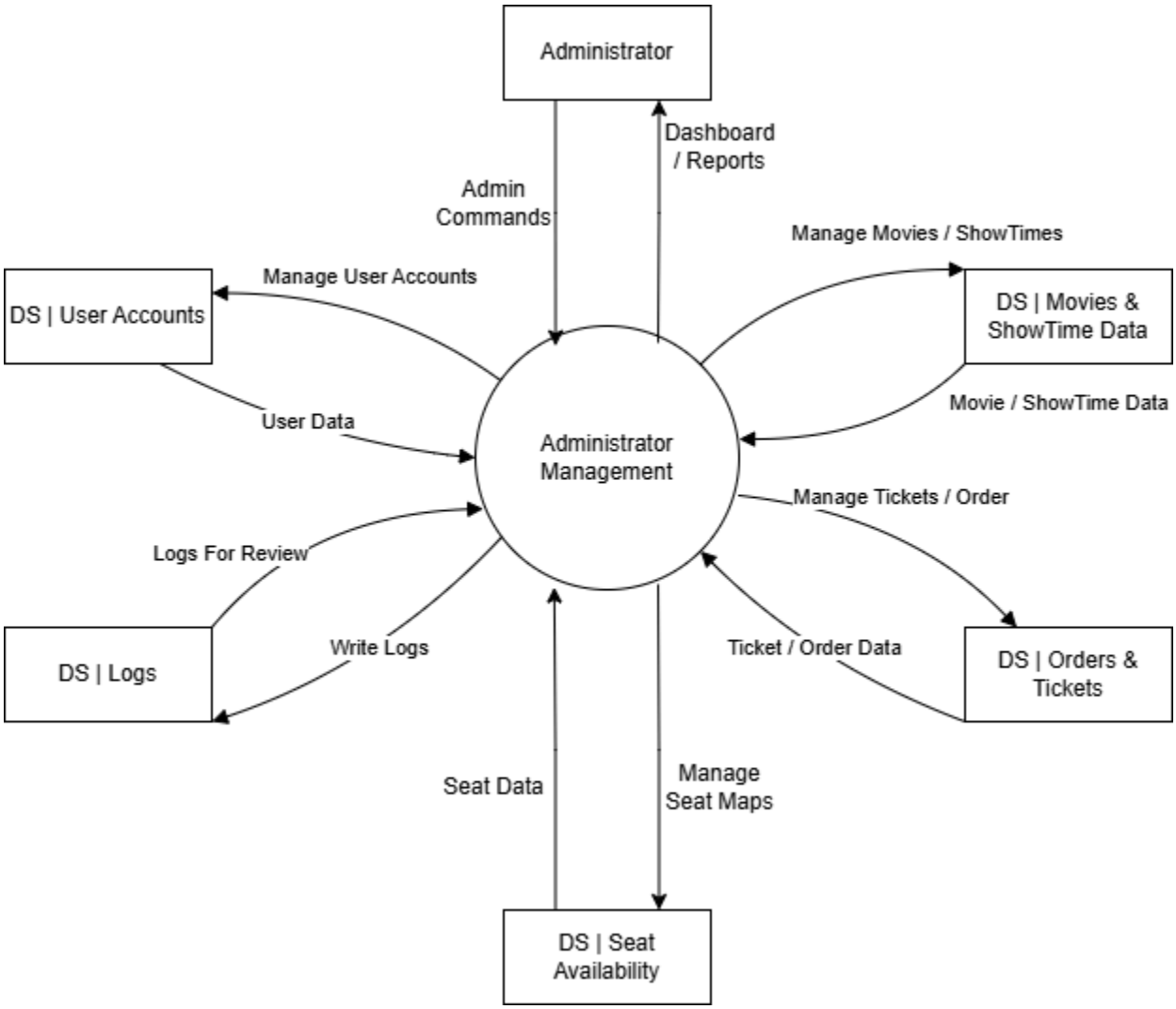
Concession Pre-Order Data Flow Diagram:

This Data Flow Diagram shows how data move through the MTWTS during the Concession Pre-order Process.



This customer submits a food order request to the concession pre-order process , which then gathers the menu and their quantities. The process then returns an order summary to the customer once they are done choosing and continues on to the payment. The payment processing then sends the payment request to the payment processor and logs the event as well as saves the order. The payment process then displays the confirmation and pickup details on the screen. While also sending another confirmation to the notification services, which then send to Email / SMS for another confirmation delivered through a secondary method.

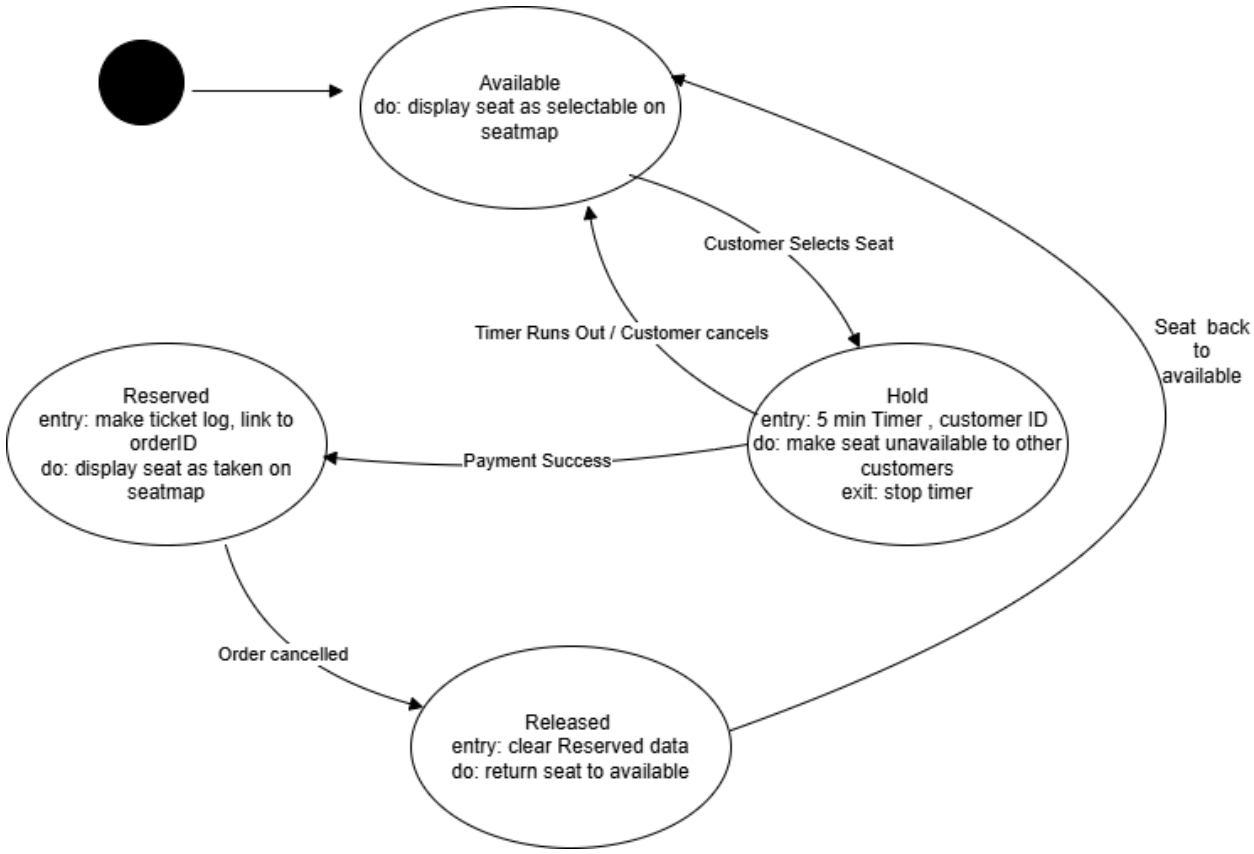
Admin Management Data Flow Diagram:
This Data Flow Diagram shows how data moves through the MTWTS during the administration management operations.



The Administrator sends commands to the administrator management process, which returns a dashboard and reports that are in the system. The process is a centralized place where the administrator can gather data from different DS such as User Accounts, Movies & Showtimes, Order & Tickets, Seat Availability, and Logs. In each of these DS except for the logs the administrator can conduct changes to the data that is contained within the DS.

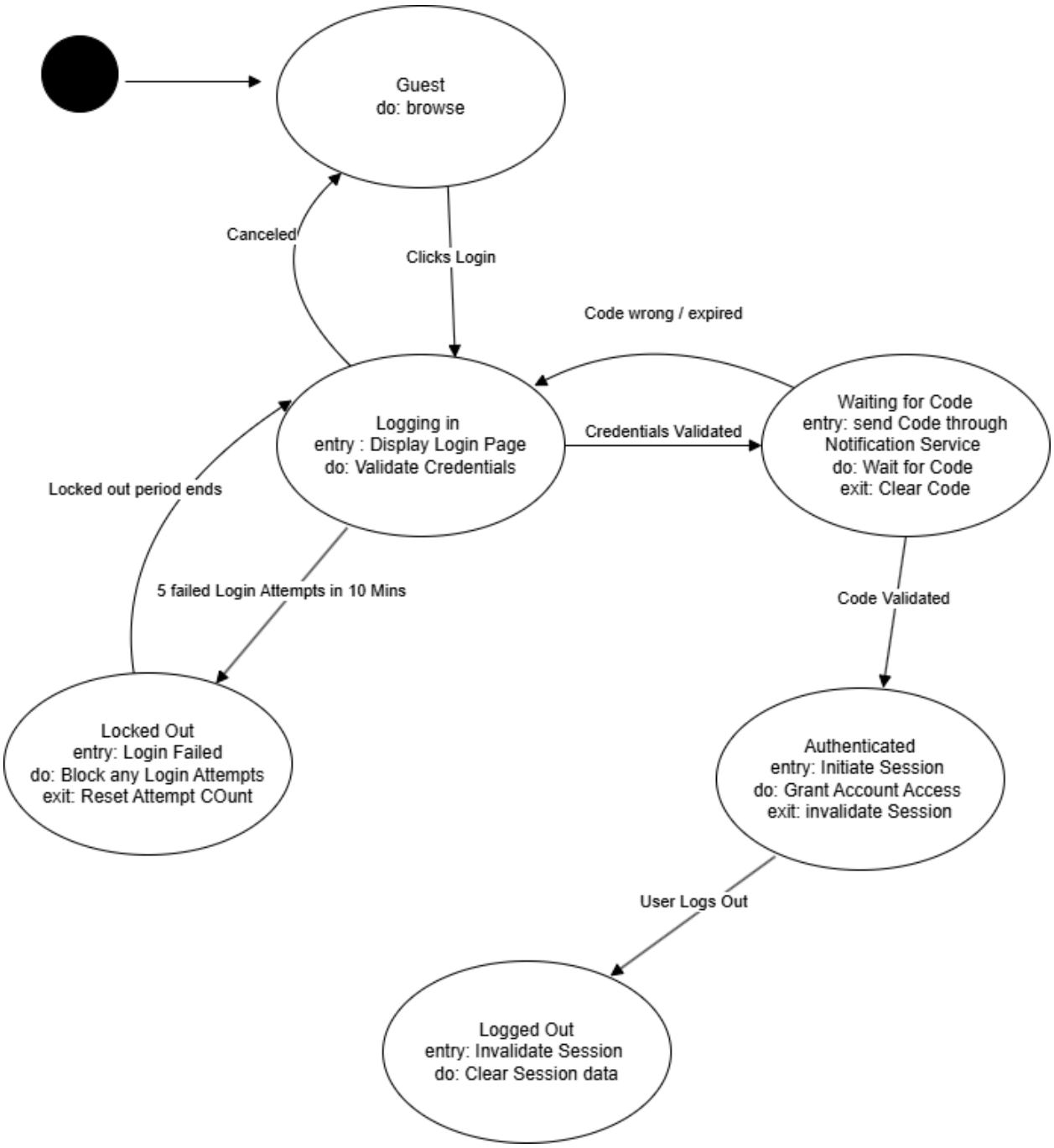
4.2 State-Transition Diagrams (STD)

Seat State-Transition Diagram:
This State-Transition Diagram models the cycle of a seat within the MTWTS from when it is available to reservation to being released again.



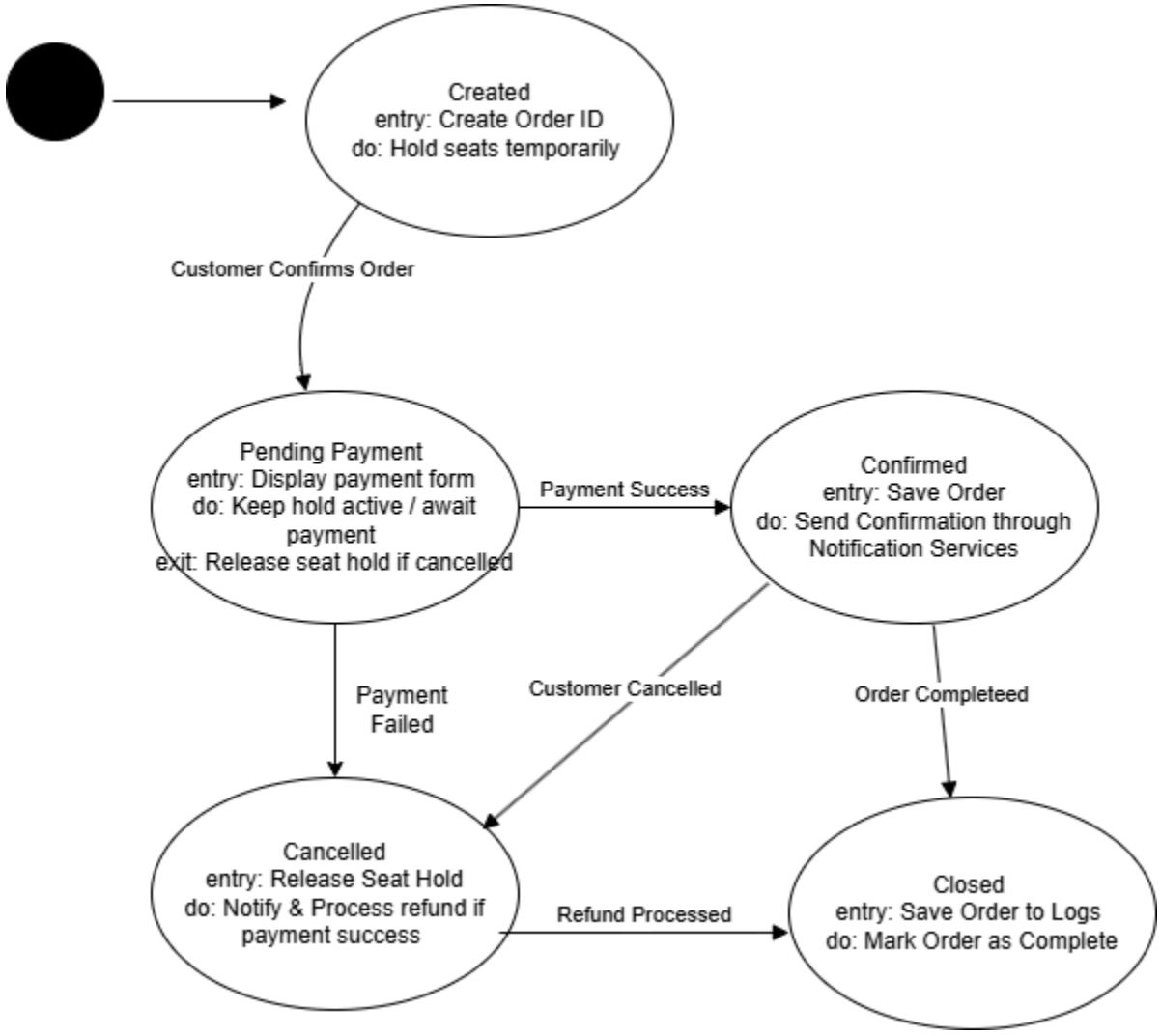
A seat begins at an available state. Once a customer selects a seat there is a 5 min timer that is started for that seat to be held. This seat then becomes unavailable to other customers. If the timer runs out or the customer cancels then the seat is once again available. If the customer succeeds in payment then the seat becomes reserved and no other customer is able to take that seat and the seat is logged with the order ID. If an order is cancelled then the seat becomes released and any reserved data is cleared and the seat is returned to being available to the next customer.

User State-Transition Diagram:
This State-Transition Diagram models the cycle of a user session within the MTWTS from the initial guest state through authentication to logout or lockout.



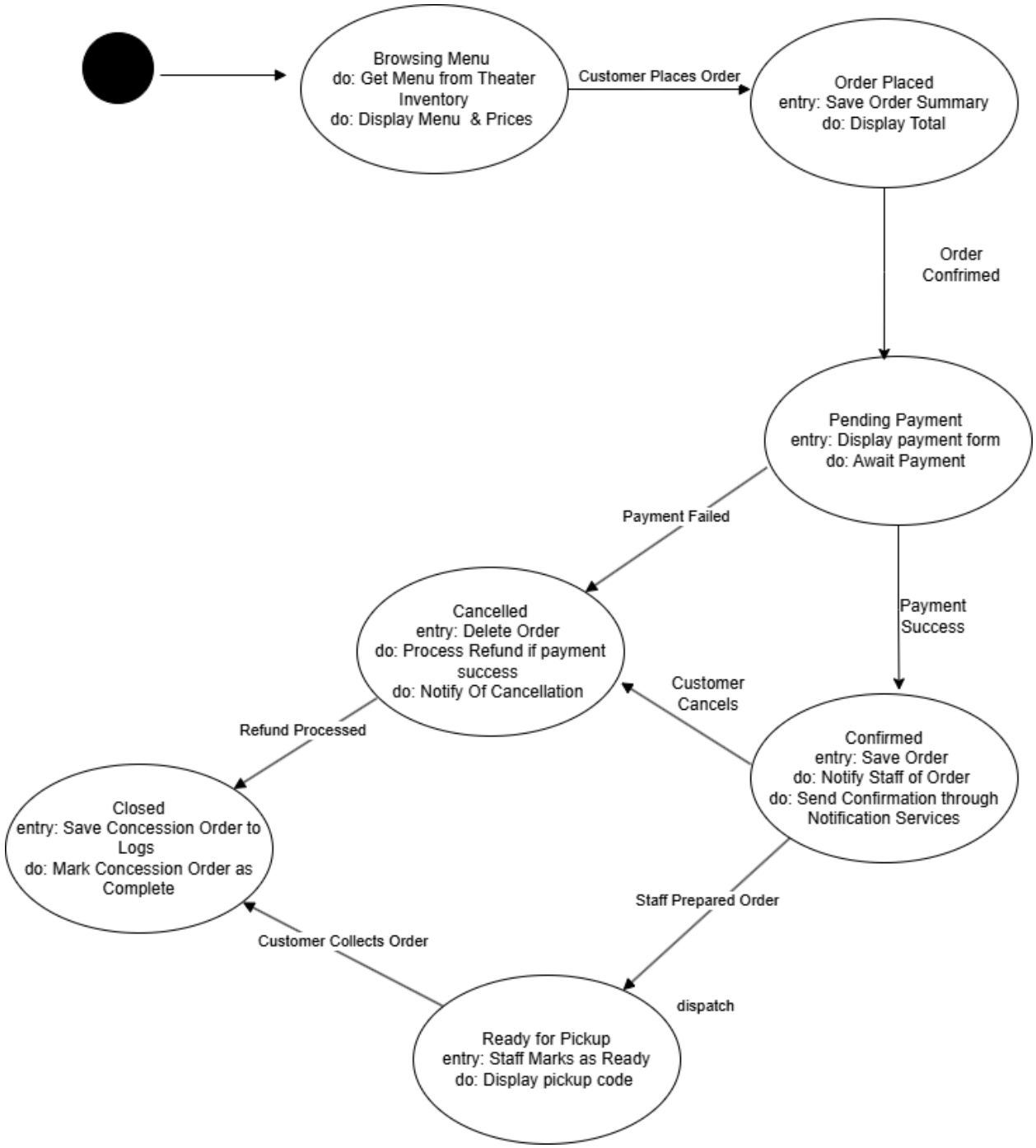
This session begins with a guest going to the login page. When the user clicks on the login they are then asked for their credentials and which if they cancel they will be brought back to being a guest. If the user fails to log in correctly within 5 attempts in 10 mins then they will be locked out of logging in until their lockout time expires. If the user succeeds then they are taken to the Two Step Verification where they are then asked to provide a code. If the code is wrong or expired then they are brought back to the log in page and asked to attempt a sign in again. If the code is validated then they are authenticated and the session is initiated. Once the user decides to logout then the session goes to being invalidated and the session data is cleared.

Order State-Transition Diagram:
This State-Transition Diagram models the cycle of an order with the MTWTS system from when it is just created to being confirmed to lastly ending up closed.



The order starts getting created by the customer. The order then holds seats temporarily and starts pending payment. Once the payment is pending and being processed it can have two situations, one where the payment is successful and the order then becomes confirmed, the order is saved and confirmations are being sent through the Notification Services. The other is when the payment is unsuccessful in which the hold on the seat is released and the customer is notified of the cancellation. If the order is confirmed and the customer decides to cancel then the order goes through the cancelled process as well, however the customer is then given a refund for the purchase that they had completed. Whether the order is confirmed or cancelled once the order is completed or the refund is processed the order then becomes closed. In a closed case the order then is saved and then is marked as completed.

Concession State-Transition Diagram:
This State-Transition Diagram models the cycle of a concession pre-order in the MTWTS system from the browsing to the order being placed to the order being closed.



The customer starts with browsing the concession menu. From there the custom can place an order in which the total is displayed as well as the summary of the order. Once the order is confirmed then the payment is pending in which case if it fails then the order is deleted and a notification is sent to the customer. If the payment is successful then the order is confined, the order is saved and the staff are notified of the new order for concessions that just came through as well as a confirmation being sent through Notification Services. If the customer decides to cancel then the order is directed to be cancelled and a processing of the payment is being completed. Once a confirmed order is being prepared by the staff the customer is then notified by the staff and prompted to display a pick up code once they get to the pickup station. Both cancelled and pickup orders are then closed , saved and marked as concessions order complete.

5. Change Management Process

This section defines how the team will request, evaluate, approve, implement, and document changes to the Movie Ticketing System throughout development. The goal is to ensure changes are controlled, traceable, and do not break previously completed functionality.

5.1 Purpose and Scope

The Change Management Process applies to any modification to:

- Requirements (functional or non-functional)
- Software design (architecture, UML classes, database schema)
- Implementation (frontend, backend, integrations)
- Testing plans, test cases, or acceptance criteria
- Deployment configuration or operational behavior

Changes may be triggered by instructor/client feedback, discovered defects, feasibility constraints, or internal improvements identified by the team.

5.2 Roles and Responsibilities

- Change Requester (any team member):
    - Creates a Change Request (CR) issue in the GitHub repository.
    - Provides rationale and affected areas.
- Design/Implementation Owner (assigned team member):
    - Performs impact analysis (scope, code, tests, data).
    - Proposes design updates and estimated effort.
- Team Lead / Coordinator (rotating weekly or designated member):
    - Schedules review discussion.
    - Ensures decisions are recorded and the documentation is updated.
- Reviewers (at least 1-2 team members not implementing the change):
    - Review the CR, approve/deny, and check design/requirements alignment.

5.3 Change Request Artifacts

All changes shall be tracked using GitHub Issues and Pull Requests.

Each Change Request (CR) issue shall include:

- CR ID: e.g., CR-001
- Title: short description of change
- Type: Requirement / Design / Implementation / Bug Fix / Refactor
- Reason: Why the change is needed
- Priority: Low / Medium / High
- Affected Components: UI, API, DB, integrations, tests, documentation
- Impacted Requirements/Design References: FR/NFR IDs, UML classes, DB tables
- Acceptance Criteria: What must be true for the change to be considered complete
- Estimated Effort: Small/medium/large (or hours)

5.4 Change Evaluation and Approval Workflow

1. Submit Request
    a. The requester creates a CR issue and tags relevant owners.
2. Impact Analysis
    a. The assigned owner evaluates:
        i. What requirements or design elements change (FR/NFR/UML/DB).
        ii. What implementation components are affected.
        iii. Risks (security, performance, usability, schedule).
        iv. Test updates needed (unit/integration/regression).
3. Review and Decision
    a. The team reviews the CR in a meeting or GitHub discussion thread.
    b. Each CR categorized as:
        i. Approved
        ii. Approved with revisions (requires modifications to plan before implementation)
        iii. Deferred (tracked but not implemented now)
        iv. Rejected (with reason documented)
4. Planning
    a. Approved CRs are added to the project board/sprint plan with an assignee.
    b. Larger changes are broken into subtasks (issues) for implementation and testing.

5.5 Implementation Controls (Branching, PRs, and Reviews)

- Each approved CR shall be implemented on a dedicated Git branch named:
    - `cr-###-short-title` (example: `cr-003-seat-hold-timeout`)
- Implementation shall be submitted via a Pull Request (PR) that includes:
    - Link to the CR issue
    - Summary of changes

    - Updated diagrams/documentation (if applicable)
    - Test evidence (screenshots/log output or test results)
- Review Requirement:
    - Each PR shall be reviewed by at least one team member who was not the primary implementer.
    - PRs must pass automated checks (lint/tests if configured) before merging.

5.6 Documentation and Traceability Updates

For every approved change, the following shall be updated as applicable:

- SRS Requirements section (FR/NFR edits, updated assumptions/constraints)
- Software Design Specification section (SWA diagram, UML diagram, class descriptions)
- Database section (schema changes, constraints, migration notes)
- Use cases / acceptance criteria (if behavior changes)

Traceability shall be maintained by referencing IDs consistently:

- Requirements: `FR-#`, `NFR-#`, `UR-#`
- Change Requests: `CR-###`
- Related PRs: include CR ID in PR title or description

5.7 Testing and Validation for Changes

All approved changes shall include validation steps:

- Unit tests updated/added for impacted modules.
- Regression testing of core flows (browse→select seats→purchase→confirmation).
- Integration testing for changes involving payment, notifications, or external data sources.
- If the change modifies business rules (ticket limits, purchase window, seat holds), update test cases and verify edge cases.

5.8 Emergency Changes (Hotfix Process)

If a critical issue is discovered (e.g., double-booking seats, security vulnerability, purchase failures):

- A hotfix branch may be created
- The fix shall be reviewed by at least one other team member when possible.
- After merging, a follow-up CR issue shall document:
    - Root cause
    - Fix summary
    - Any additional preventative steps (tests, monitoring)

5.9 Versioning and Release Notes

- Each significant update shall increment the document version on the title page or revision history section (if present).
- Release notes shall include:
    - Implemented CR IDs

    - Summary of new features/bug fixes
    - Any known limitations introduced or resolved

This change management process ensures controlled evolution of the requirements, design, and implementation, while maintaining traceability and quality across the group project.

6. Test Plan (Verification & Validation)

6.1 Purpose and Scope

The Test Plan defines verification and validations activities to ensure that the Movie Theater Web Ticketing Systems meets its functional requirements set in Section 3.2, non-functional requirements set in Section 3.5, and other requirements set in Section 3.9.
Testing covers the customer browsing and ticket purchase flow, discount pricing, seat-hold/inventory correctness, bot protection, admin access, and notification/receipt delivery.

6.2 Test Strategy and Levels

1. **Unit Testing:** Validates individual functions/classes in isolation using mocked dependencies like the DB, payment processor, email/SMS, review sources).
2. **Integration Testing**: Validates interactions between services and their workflow connections (API ↔ DB, Order ↔ Payment, Order ↔ Inventory, Auth ↔ Notification, Bot Protection ↔ Logging).
3. **System Testing:** Validates complete end-to-end user journeys in an environment that resembles production.

6.3 Test Environment

- Client: Modern browsers (Chrome/Firefox/Safari/Edge) per UI requirement.
- Server: Test deployment of backend services + relational DB.
- External Integrations: Payment processor sandbox, email/SMS test channel; review source mocked or sandboxed.
- Seed Data: Movies, showtimes, auditorium seat maps, discount ticket types.

6.4 Test Data/Vectors

- Showtimes:
    - S1 = 7 days from now (valid)
    - S2 = 30 days from now (too early)
    - S3 = showtime started 20 minutes ago (too late)
- Seat map sample: rows A-E with at least 50 seats to validate holds and conflicts
- Discount types: adult, child, student, senior
- Bot vectors: high-rate traffic bursts from single IP/account, repeated checkout attempts

6.5 Test Sets

6.5.1 Unit Test Sets

UT-Set A: Pricing and Discount Logic
- Target: Ticket price calculations and discount application (FR-2.7, FR-2.8)
- Coverage: Valid discounts, invalid discount type, rounding/precision, tax/fee inclusion
- Failure Modes Covered: Incorrect total, discount not applied, negative totals.

UT-Set B: Seat Hold Timer + Release
- Target: Seat hold creation and expiration behavior (SW-1.2, NFR-REL-1).
- Vectors: Hold timeout boundary (exactly 5 min), cancellation before timeout, multiple seats held in one order.
- Failure modes covered: seat remains stuck in "Held", seat double-allocated, timeout not enforced.

6.5.2 Integration Test Sets

IT-Set A: Purchase Transaction Integrity
- Target: Order ↔ Inventory ↔ Payment ↔ Notification
- Requirements: FR-2.6, EH-2.1, OR-1.1, OR-1.2
- Vectors: payment success, payment decline, payment timeout, DB write failure simulation

IT-Set B: Bot Protection and Logging
- Target: Security/Bot protection ↔ Logging
- Requirements: FR-3.1–FR-3.5
- Vectors: 100 requests/60 sec same IP, repeated failed checkout, CAPTCHA triggered/unavailable.
- Failure Modes Covered: No rate limiting, no verification, missing log entries.

6.5.3 System Test Sets

ST-Set A: End-to-End Ticket Purchase Flow (Happy Path + Rule Boundaries)
- Target: Full customer flow from browse → showtime → seat selection → checkout → confirmation
- Requirements: FR-1.1–FR-1.4, FR-2.1–FR-2.6, UI-1.4
- Vectors:
    - Valid showtime within window
    - Attempt 21 tickets (must block this)
    - Attempt too early/too late purchase window

- Failure Modes Covered: Rule checks not enforced, incorrect messaging, missing confirmations.

ST-Set B: High-Demand and Availability Behavior
- Target: Performance under load and graceful degradation
- Requirements: NFR-PERF-1, NFR-PERF-2, NFR-PERF-4
- Vectors: 1000 concurrent users browsing, seat map load, checkout attempts
- Pass Criteria: System remains responsive and uses queue/rate limit messaging instead of crashing

6.6 Traceability

Each requirement in scope shall be traceable to at least one test case ID. The Excel sheet "Team-5_Test-Cases" (see repo link below) contains requirement mappings per test case.

6.7 Defect Reporting

Defects shall be logged as GitHub issues with: steps to reproduce, expected vs actual output/results, screenshots/logs, severity, and affected requirement IDs.

6.8 Github Links

Repository Links: https://github.com/miriamvalenzuela/group-5-test-plan.git
https://github.com/miriamvalenzuela/group-5-srs.git
Excel Test Cases: https://github.com/miriamvalenzuela/group-5-test-plan/commit/f64c50c7bF6155317815624a014bc208a9c5eade
Architecture Design with Data Management Strategy:
https://github.com/miriamvalenzuela/group-5-srs.git

7. Data Management Strategy

7.1 Overview

The Movie Theater Web Ticketing System is data-driven and persists both operational and sensitive data. The data strategy focuses on:
- Correctness and integrity
- Security
- Availability and performance under high demand
- Traceability and auditability via logs.

7.2 Data Store Selection and Rationale (SQL vs NoSQL)

We chose one operational relational (SQL) database for the system's main data.

We picked SQL because ticketing needs strong rules like:

- One seat can't be sold twice for the same showtime
- Orders and payments need to stay consistent (either the order succeeds and seats are sold, or it fails and seats go back to their previous availability)

SQL supports these transactions and integrity constraints that reduce risk of inconsistent inventory or order state.
An external Theater Inventory/Showtime DB (the theater's system) to get showtimes/seat map data, will be connected, therefore no orders, holds, users, etc. will be stored in this DB and instead will only be kept in the operational SQL database along with the system;s main data.

7.3 How Data Is Split Logically

The operational database is organized by domain to keep responsibilities clear:

A) Identity & Access

- Users (registered users, admin accounts), authentication artifacts, session metadata (where applicable)

B) Catalog & Scheduling

- Movies, showtimes, auditoriums, seat maps (may be synced from external theater DB)

C) Inventory & Seat Holds

- Seat instances per showtime and their states (Available/Held/Sold)
- Seat hold records (who holds it, hold expiration)

D) Commerce

- Orders (ticket orders + optional concession orders)
- Tickets (ticket type/discount applied)
- Payments (token/outcome/status only; never raw card details)

E) Feedback & Observability

- Customer feedback submissions
- Logs (security, payment failures, admin actions, bot events)

7.4 Data Ownership and Source of Truth

The MTWTS manages its internal database as the primary source of truth for the user generated records.

MTWTS Internal Data Ownership

- User Accounts - User Profiles, Admin Profiles
- Orders - Purchase History, Order Status, Pricing
- Tickets - Tickets Bought, Ticket Status
- Seat Holds / Seat Availability - Temporary Reservations, Hold Timers, Real Time Availability
- Logs - Security Events, Admin Actions

The MTWTS uses the Theaters database as the source for any theater related information

Theater

- Seat maps - Physical Location of all the seats in their given auditorium , showing row & numbering as well as special seat designations
- Movie Information - Includes the movie showtimes, start times, runtime & specified auditorium

The MTWTS has external third party providers that handle the processing and records for their specific operation

External

- Payment Transactions - Payment Processor
- Email /SMS notifications - Notification Service
- Authentication - Identifies Users

7.5 Consistency and Transaction Strategy

The MTWTS system uses SQL to ensure that data is consistent especially during high traffic times, the system makes use of a strategy to ensure data integrity.

Transaction Strategy:

Tickets sales are made through one SQL atomic transaction ensuring that all the actions are successful and continue or unsuccessful and roll back.

- Atomic: During transactions to make sure that they are successful they must have an Order, the Ticket generated, Seats go from holding to sold, and payment is processed.
    - In case of Payment failure or the transaction is discarded , the entire transaction will roll back and the seats will go from held to available seats
- Seat holds ensure that seats become temporarily unavailable to other users upon the selection of the seat

- This prevents the double selling of seats, as only one user is allowed to hold a seat per allotted time

7.6 Sensitive Data Handling

The database system protects and limits access to sensitive data, with emphasis on payment information and personal user data otherwise known as personally identifiable information (PII).

Sensitive data falls into three tiers:

- **Tier 1 Never Stored:** Data such as raw payment card data entailing full card numbers, CVVs, expiry dates. This type of payment data if exposed in a data breach would expose usable financial credentials. The system handles payment processing through a thor party. The only thing stored is a payment token and outcome, logging that a payment occurred.
- **Tier 2 Encrypted:** Personally Identifiable Information (PII) entailing full names, email addresses and phone numbers. These are stored within the database but encrypted at rest using a secure symmetric encryption that meets industry standards. Encryption keys are stored on another database that is not publicly accessible and in a dedicated secrets manager. Thus even should the database be breached and contents leaked they cannot be used.
- **Tier 3 Access Controlled:** Data such as order history, ticket records and seat assignments. These are protected by access controls and database level encryption at rest but individual fields are not separately encrypted.

Secure data stored is to follow these main policies:

A) **Password Handling:** Passwords are never ever stored in plaintext or use an outdated encryption. Passwords are hashed using an industry standard encryption algorithm and salted. Should the users table ever be leaked passwords cannot be directly read.

B) **Minimum Data:** The cistern will only collect data that is necessary for operations. Date of birth may be necessary in order to purchase movies rated R18, but only a date of birth would need to be specified, and full IDs would not be required to be submitted but rather a physical process that occurs at theaters. Less data is a smaller attack/liability surface should a data breach occur.

C) **Secure Logging:** All logs with regards to security or payment recorded events are cleaned of any sensitive values before being written to ensure that should logs be read by a malicious actor they cannot act as an attack vector.

7.7 Access Control and Network Boundaries

The system ensures the access control with authentication of the user , confirming that the user cannot go beyond their restricted access.

Access Control

The system ensures the access control with authentication of the user , confirming that the user cannot go beyond their restricted access.

- Customer - Browsing Movies & Showtimes, Booking Tickets, Concessions, Refund Requests (No User can access another Users information)
- Standard Admin - Managing Movie Showtimes and records.
- System Admin - Full access of Standard Admin , including User Management & Logs

Network Boundaries

The system enforces strict network boundaries to ensure that no sensitive information and internal components are exposed to the public

- Public Zone: The public is only allowed on the Web Frontend through HTTPS
- Internal Zone: Database is not publicly accessible and can only be accessed through secure internal channels that reject any incoming traffic except for authorized requests
- External Zone: Calls to any external service (Payment , Notification) are done through exclusive zone boundary that only dedicated services (such as booking for payment) are allowed to make , ensuring that communication is logged

7.8 Retention, Archival, and Deletion

The system enforces different life spans (how long the data is kept in long term storage) based on the type of data.

- **Retention based on data domain**
    a. Session and authentication artifacts are deleted within 24 hours of expiry or logout since they have no long term value.
    b. Seat hold records are deleted immediately once a hold expires or converts to an order.
    c. Order and ticket records are retained in an active database for at least 1 year in order to support possible disputes, refunds or customer inquiries.
    d. Payment records are retained for a minimum of 1 year and then deleted unless suspicious activity is detected.
    e. Security and audit logs are retained for at least 6 months and kept in hot storage for active monitoring and incident response.

Deletions follow a soft delete pattern where a deleted time stamp is set on the record before any permanent removal happens. This way there is a grace period before hard deletions happen via a scheduled background job to prevent accidental permanent data loss. Security logs are appended only and cannot be modified or delayed through the UI by any user including administrators.

7.9 Backups and Recovery

The operational SQL database follows a layered backup schedule.

- Full backups are taken over every 24 hours during off peak hours and exported to offsite storage in a separate region from the primary database.
- Incremental backups are taken every hour to capture all changes since the last full backup and limit the maximum data loss window to 1 hour.
- Full backups are retained for 30 days and incremental logs are retained for 7 days.

Recovery happens by restoring the most recent full backup and replaying incremental logs until the point of failure. Restore procedures are well documented and tested periodically in a staging environment. The recovery objectives are the following:

- The recovery point objective of approximately 1 hour means at most 1 hour of transactions could be lost in the worst case failure scenario.
- Recover time objective of approximately 2-4 hours meaning a full restore from the latest backup is to be completed within that window.

This backup strategy helps to ensure a 99.9% uptime requirement by ensuring data loss events can be recovered from within an acceptable window.

7.10 Schema Evolution (Migrations)

Database schema changes are managed through versioned migrations (e.g., migration scripts tracked in GitHub).
Changes are reviewed via PRs and tied to Change Requests (CR-*###*) per the Change Management Process.

7.11 Alternatives Considered and their Tradeoffs

Alternative 1: NoSQL database for everything

- **Pros:** flexible schema, scales easily
- **Cons:** harder to guarantee “no double-selling” and atomic order updates without extra work

Alternative 2: Multiple databases (one per service)

- **Pros:** good isolation, services can scale independently
- **Cons:** more complicated to keep orders + inventory consistent (distributed transactions/eventual consistency issues)

Why our choice is reasonable:
A single SQL database is simpler for a class project and fits ticketing well because correctness is more important than fancy scaling. If we needed to scale later, we could add caching or read replicas without changing the core data model.

A. Appendices

A.1 Appendix 1

Status: Informative only (not part of the binding requirements).
This appendix contains summarized notes from in-class client interviews (Group 5, Feb 12, 2026). Notes include:

- User goals and pain points for buying tickets online
- Desired features and priorities (movie browsing, showtimes, seat selection, discounts)
- Constraints and business rules (purchase window and ticket maximum)
- Admin needs (showtime management, reporting/log review)
- Assumptions and open questions identified during interviews

Purpose: Provides traceability and justification for functional requirements and use cases in Section 3, especially FR-1 through FR-5 and UC-1 through UC-5.

A.2 Appendix 2

Status: Informative only (not part of the binding requirements).
This appendix maps key requirements to the artifacts that verify them. At minimum, include a table with columns like:

- **Requirement ID** (FR/NFR/OR)
- **Related Use Case(s)** (UC-#)

- **Related Analysis Model(s)** (Sequence / DFD / STD)
- **Verification Method** (Test / Demonstration / Inspection)
- **Notes**

Example entries:

- FR-2.1 (20 ticket limit) → UC-1, UC-5 → STD (Order), Sequence (Purchase) → Test
- NFR-PERF-1 (2s search) → UC-1 → DFD (Browsing) → Performance test
- FR-3.1 (rate limiting) → UC-1/UC-2 → DFD (Purchase flow), Logs → Test/Inspection